



제14장 신경망III (딥러닝)

학습 목표

- 심층 신경망(DNN: Deep Neural Networks) 구조를 이해한다.
- 은닉층의 역할을 이해한다.
- 그래디언트 소실 문제를 이해한다.
- 새로 등장한 여러 가지 활성화 함수를 이해한다.
- 다양한 손실 함수를 이해한다.



01 딥러닝

- 심층 신경망(DNN: Deep Neural Networks)은 MLP(다층 퍼셉트론)에서 은닉층의 개수를 증가시킨 것이다.
- "딥(deep)"이라는 용어는 은닉층을 여러 개 사용하는 것을 의미한다.
- 최근에 딥러닝은 컴퓨터 시각, 음성 인식, 자연어 처리, 소셜 네트워크 필터링, 기계 번역 등에 적용되어서 인간 전문가에 필적하는 결과를 얻고 있다.

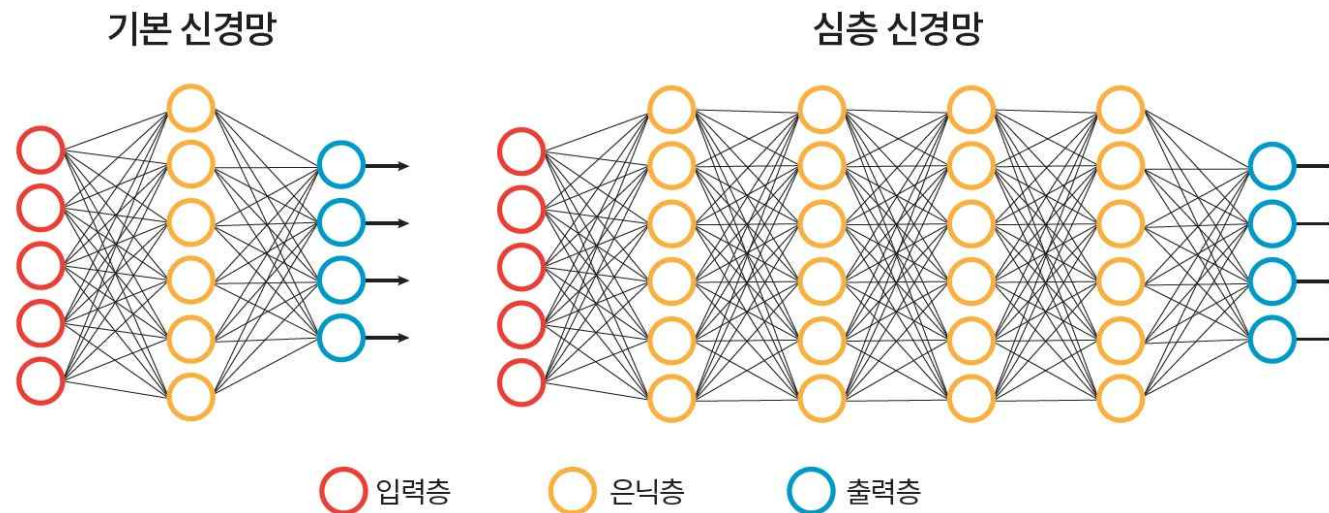


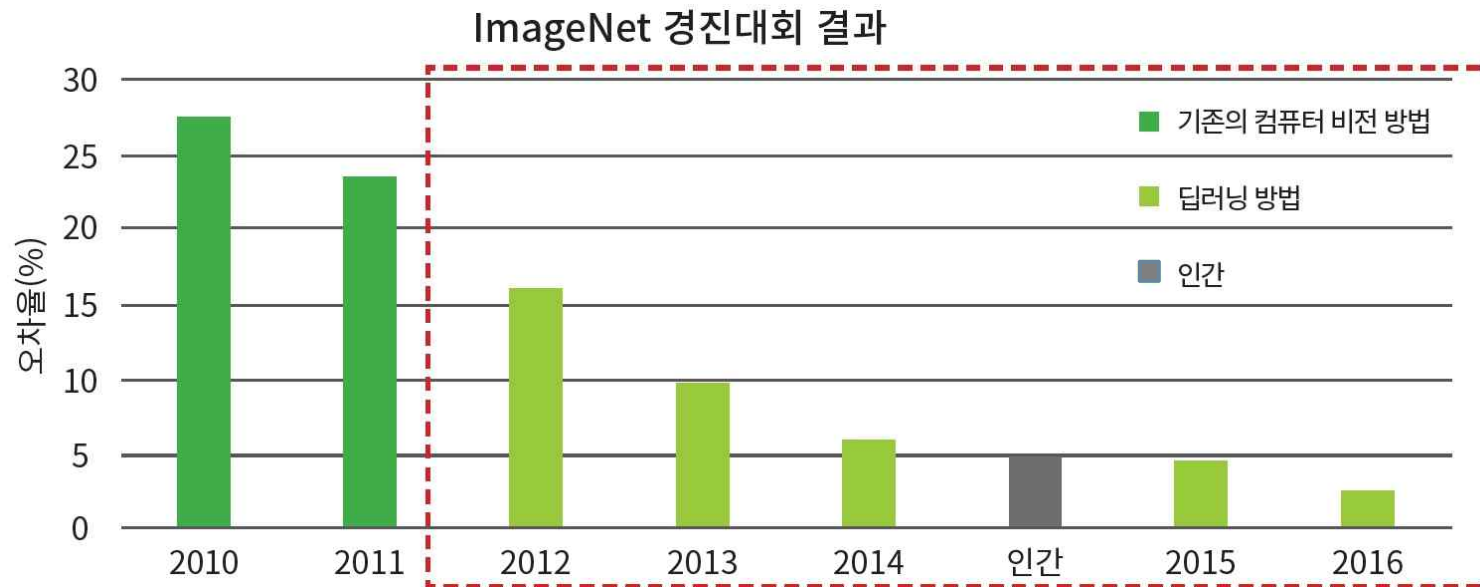
그림 14-1 일반 신경망과 DNN

딥러닝은 MLP의 여러 가지 문제점을 효과적으로 해결

- 그래디언트 소실(**gradient vanishing**) 문제: 은닉층이 많아지면 출력층에서 계산된 그래디언트가 역전파되다가 값이 점점 작아져서 없어진다.
 - Sigmoid -> ReLU
- 과잉 적합(**over fitting**)
 - 과잉 적합이란 훈련 데이터는 잘 분류하지만, 새로운 데이터가 들어오면 분류가 잘되지 않는 현상이다.
 - Drop Out
 - Regularization
- 또 은닉층이 많아지면 계산 시간이 과다해지는 문제점이 있었다.

2012년 ImageNet

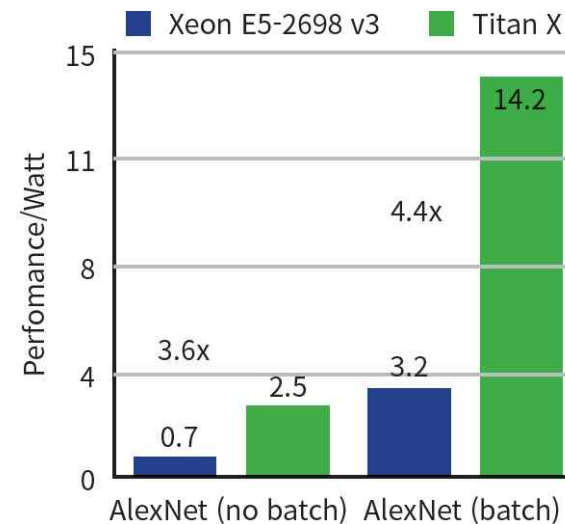
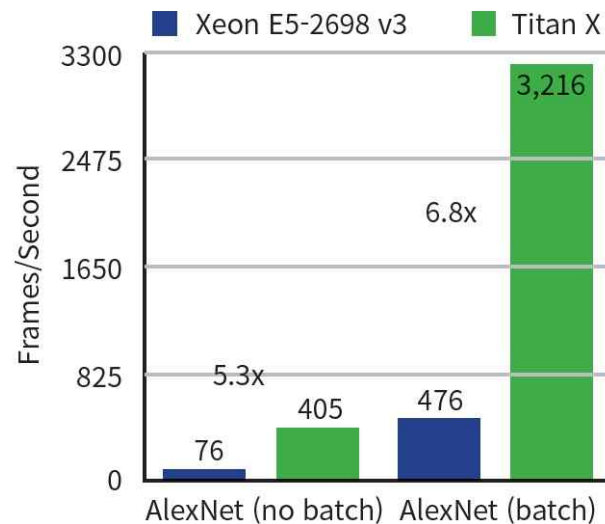
- 2012년에는 Krizhevsky 등이 만든 딥러닝 시스템 AlexNet이 다른 기계 학습 방법들을 큰 차이로 물리치고 ImageNet 경진대회에서 우승한다



GPU(Graphic Processor)의 도움

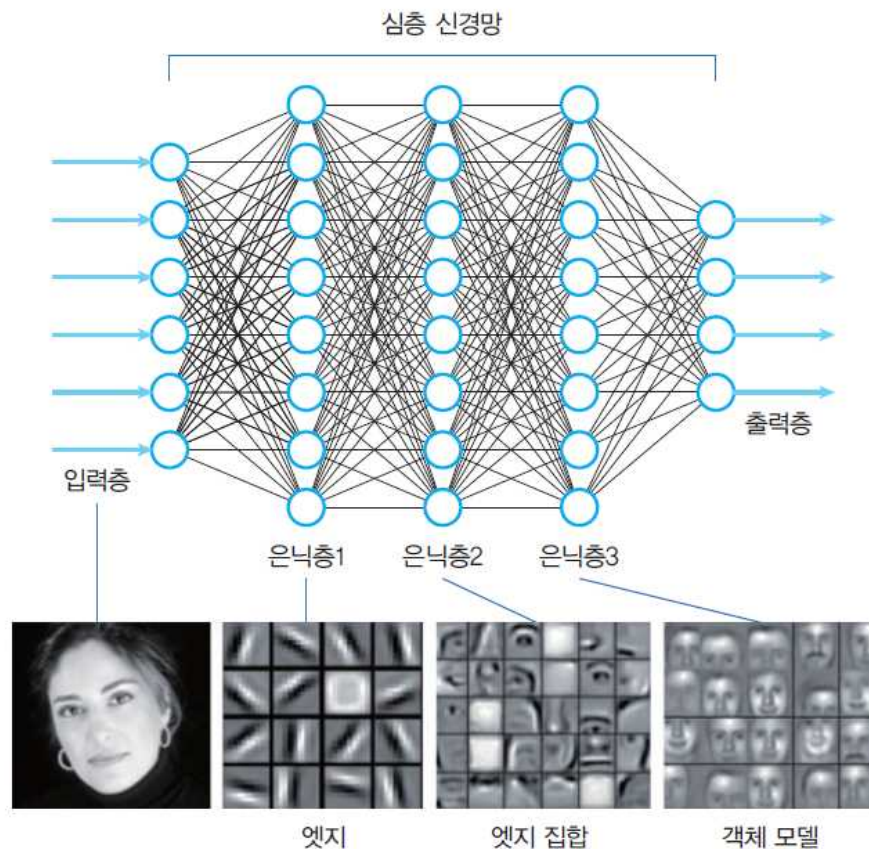
- DNN의 학습 속도는 상당히 느리고 계산 집약적이기 때문에 학습에 시간과 자원이 많이 소모된다.
- 최근 GPU(Graphic Processor) 기술이 급속히 발전하면서 GPU가 제공하는 데이터 처리 기능을 딥러닝도 사용할 수 있게 되었다. 딥러닝 혁명에는 게이머들의 도움도 컸다.

CPU와 GPU의 성능비교



은닉층의 역할

- 은닉층은 무슨 역할을 하고 있는 것일까? -> 여러 개의 은닉층 중에서 앞단은 경계선(에지)과 같은 저급 특징들을 추출하고 뒷단은 코너와 같은 고급 특징들을 추출한다.



MLP vs DNN

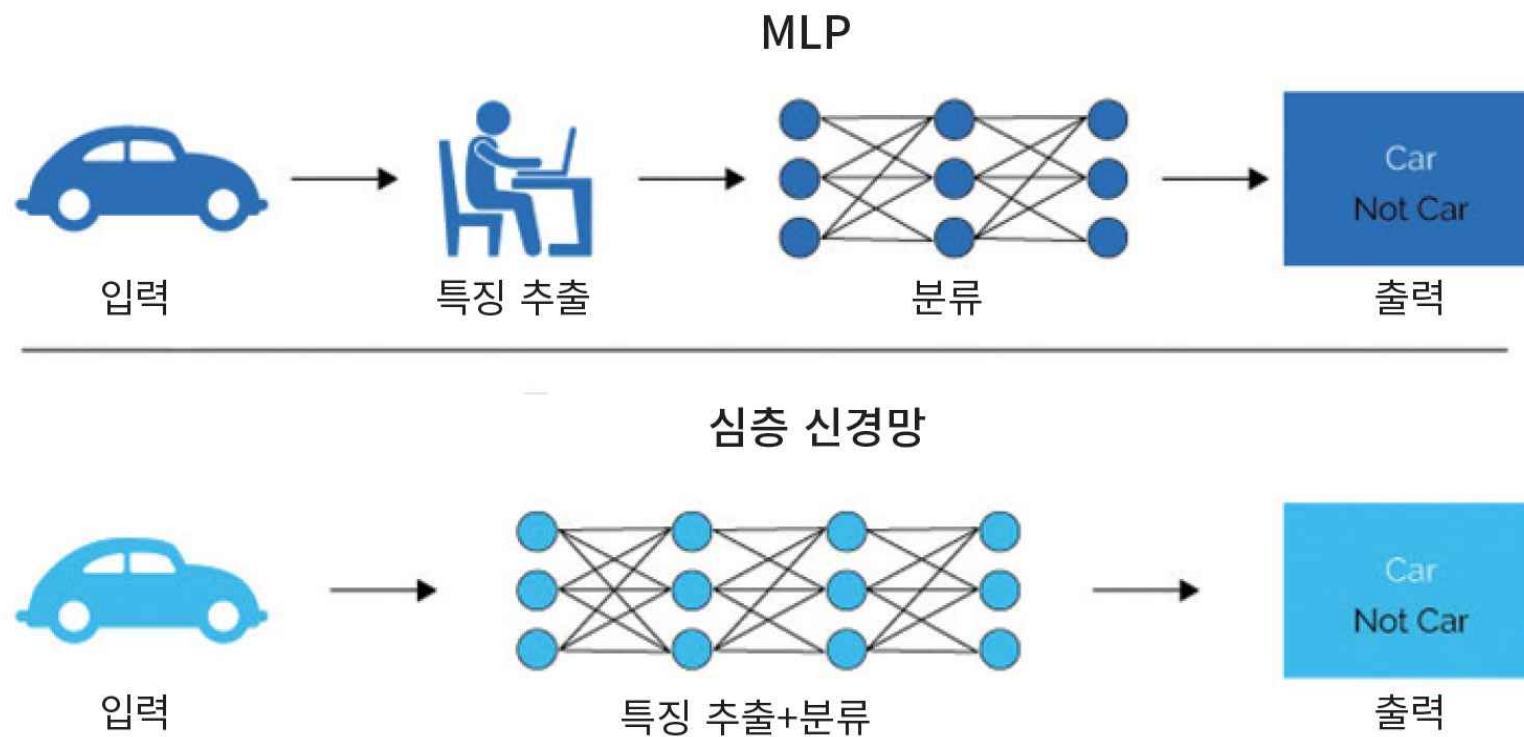
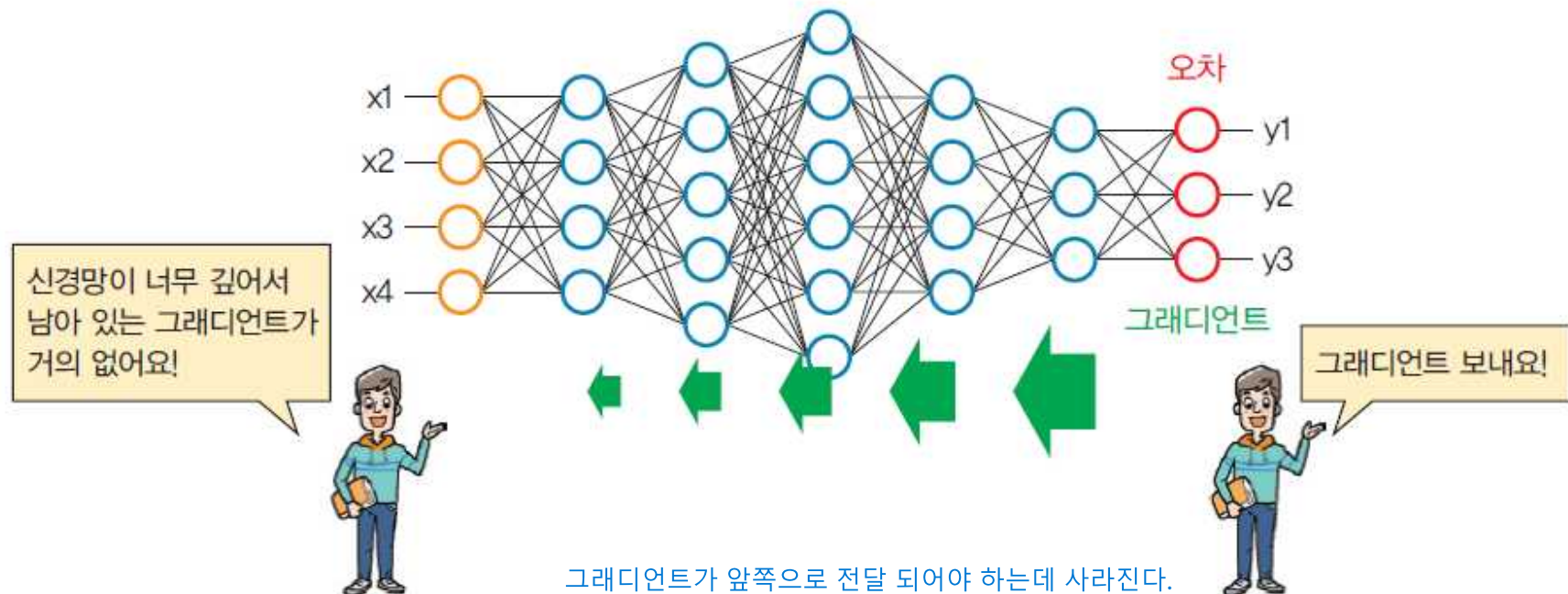


그림 14-3 MLP와 심층 신경망의 비교

그래디언트 소실 문제

- 심층 신경망에서 그래디언트가 전달되다가 점점 0에 가까워지는 현상이다



그래디언트 소실 문제

- 범인은 **시그모이드 활성화 함수**
- 시그모이드 함수의 특성상 아주 큰 양수나 아주 큰 음수가 들어오면 출력이 포화되어서 이 부분의 기울기는 거의 0이 된다.

시그모이드 활성화 함수가 문제였다.

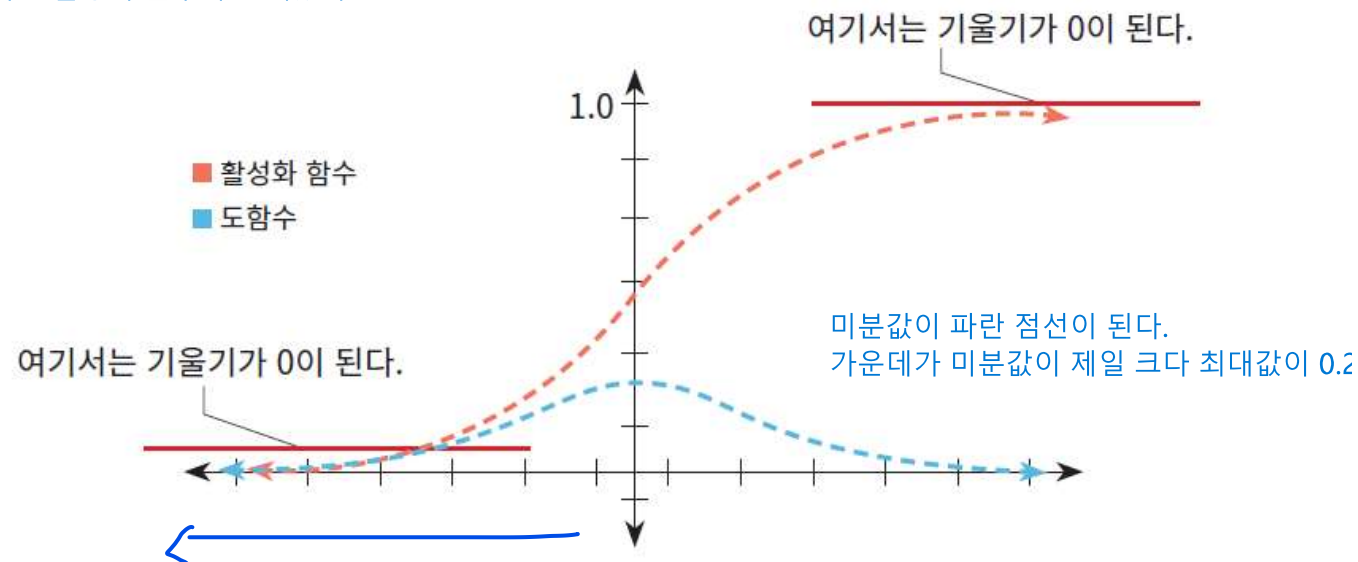


그림 14-6 시그모이드 함수와 그래디언트 소실

앞쪽으로 가면 파란 점선값이 그대로 전달이 된다.

그래디언트 소실 문제

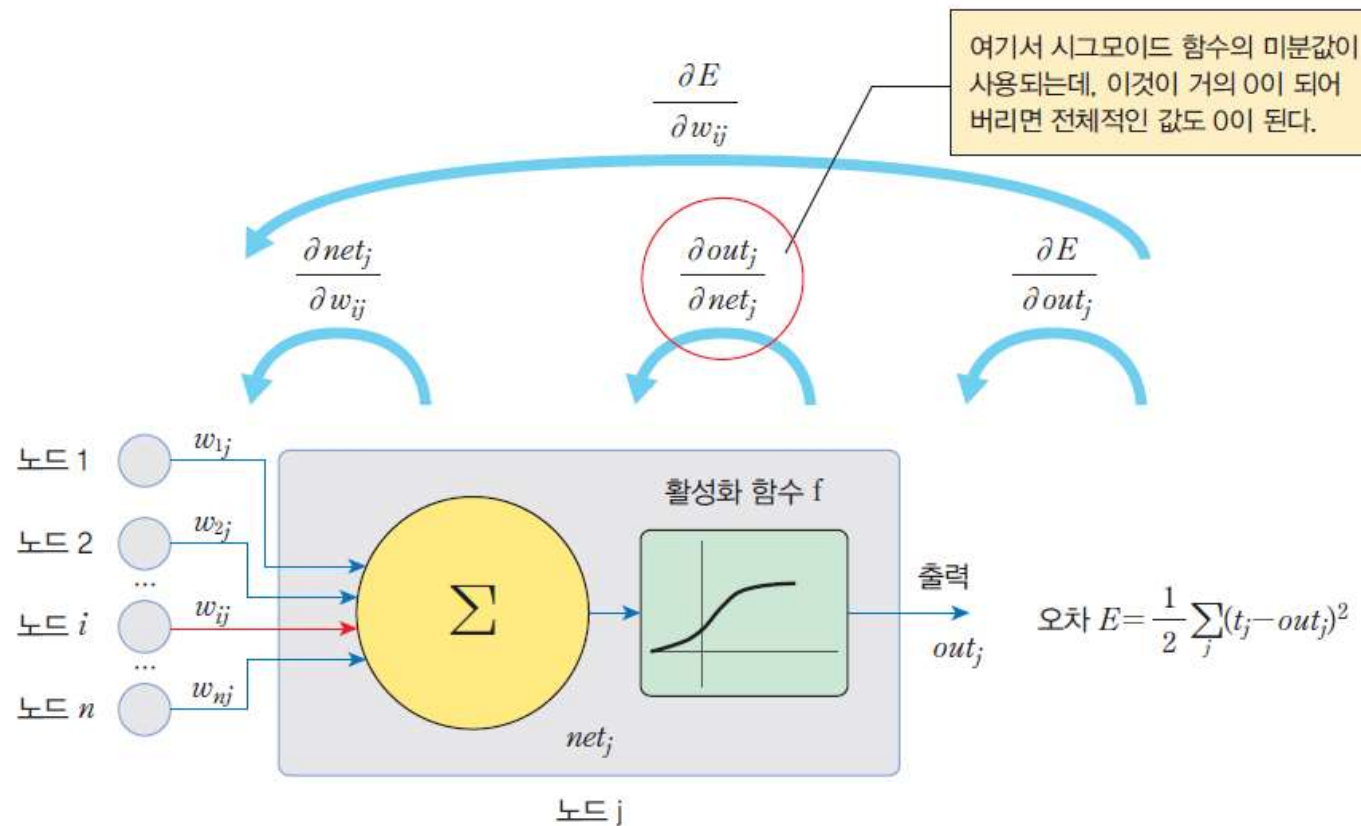
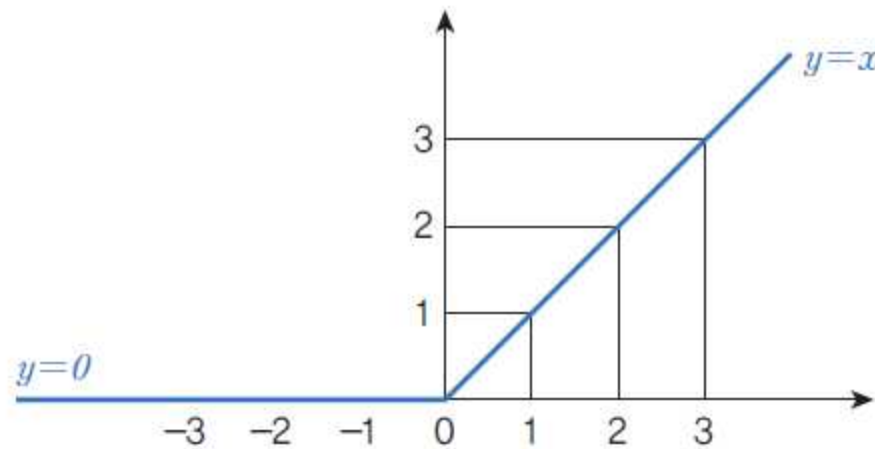


그림 8-6 그래디언트 소실 문제

새로운 활성화 함수

- 그래디언트 소실 문제를 해결하기 위하여 심층 신경망에서는 활성화 함수로 ReLU 함수를 많이 사용한다.

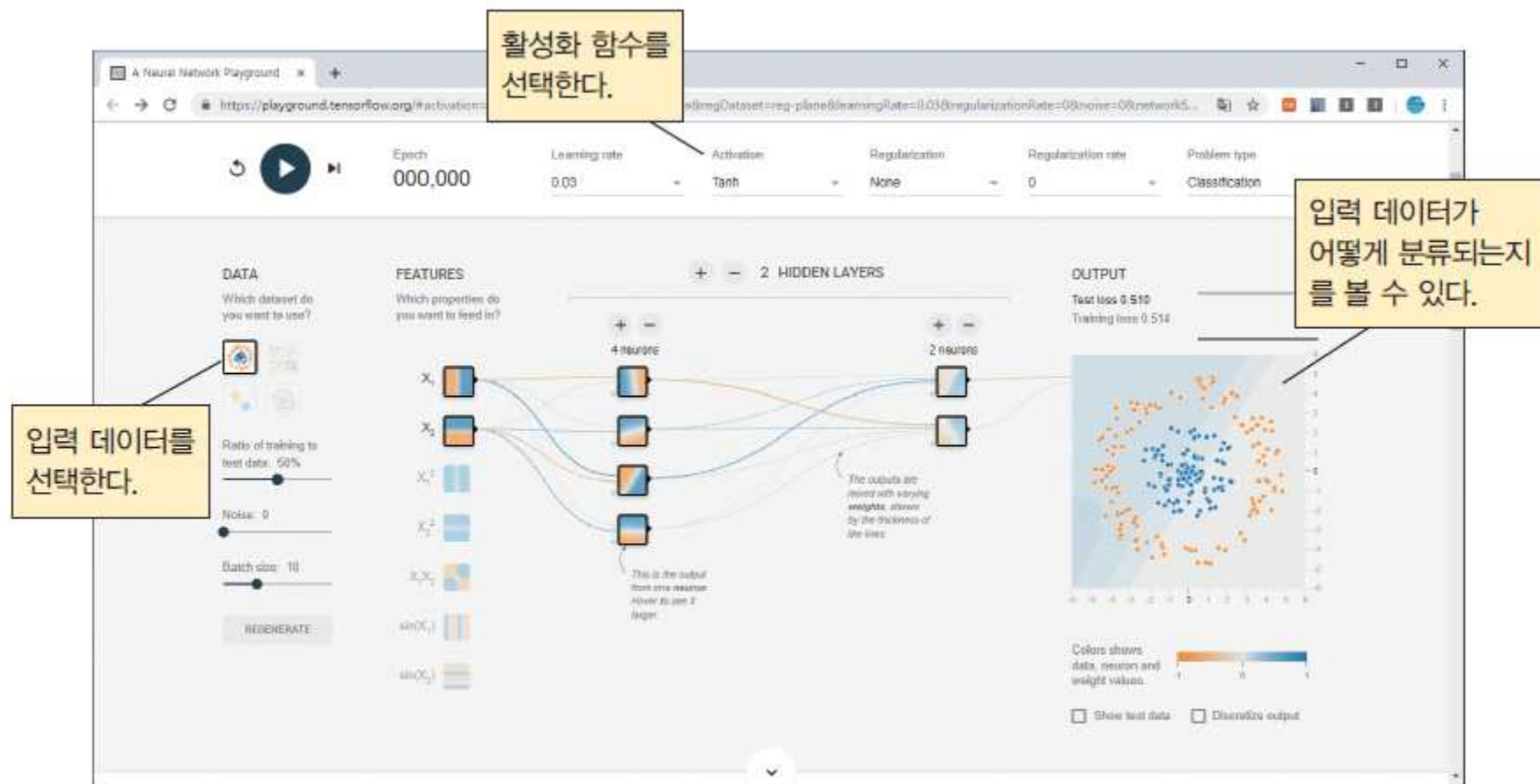


$$f(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

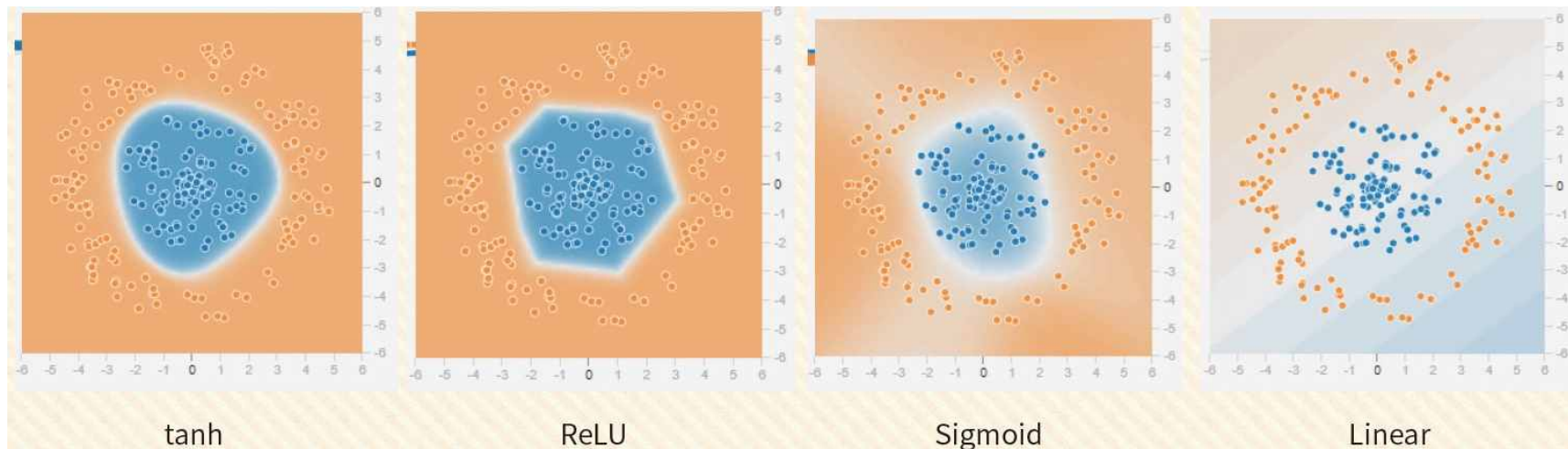
그림 8-7 ReLU 함수

Lab: 활성화 함수 실험

- <https://playground.tensorflow.org>로 실험



Lab: 활성화 함수 실험



Q

- 은닉층이 많아지면 출력층에서 계산된 그래디언트가 역전파되다가 값이 점점 작아져서 없어지는 문제점을 무엇이라고 하는가? 그리고 이것의 원인은 무엇인가?

그래디언트 소실 문제, 원인은 활성화 함수 시그모이드
이걸 해결하기 위해 relu를 사용해서 해결하였다.

03 손실 함수 문제

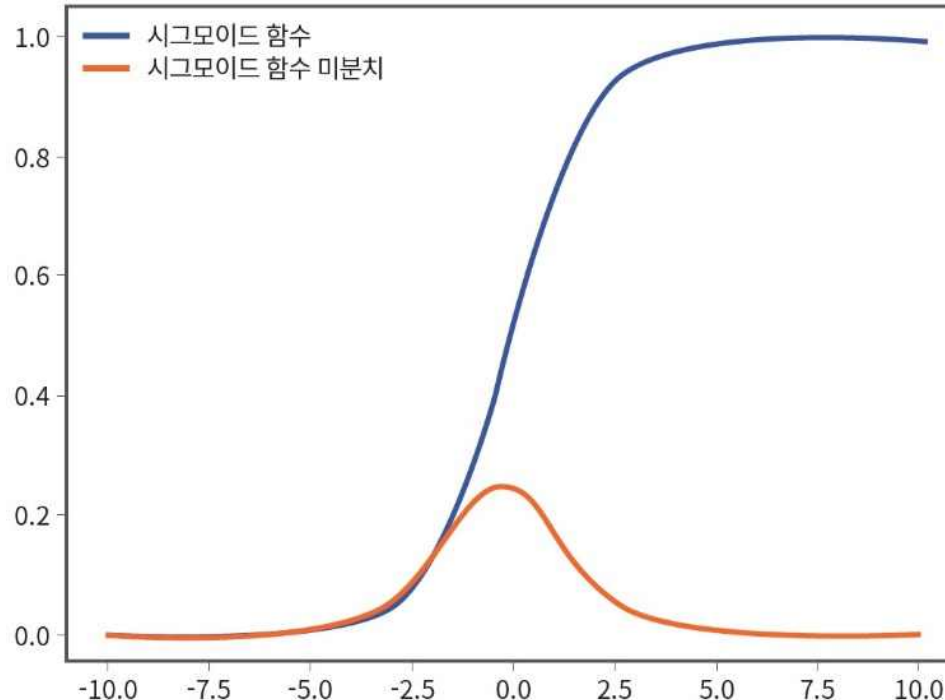
- 손실 함수로 이제까지는 평균 제곱 오차 함수(Mean Squared Error: MSE)를 사용

$$E = \frac{1}{2n} \sum_{i=1}^n (t - o)^2$$

- 출력노드의 활성화 함수로 시그모이드(sigmoid)가 사용된다면 MSE는 저속 수렴 문제(slow convergence)에 부딪치게 된다.
 - 시그모이드 함수의 특성상 미분했을 때 0 근처에서는 미분값이 크고 멀어질수록 급격하게 미분값이 0에 가깝게 된다.
 - 즉, 입력값(오차)이 작을 때는 과다하게 학습하고, 큰 입력값(오차)에는 학습이 작게 이루어지는 모순이 발생한다.

손실 함수 문제

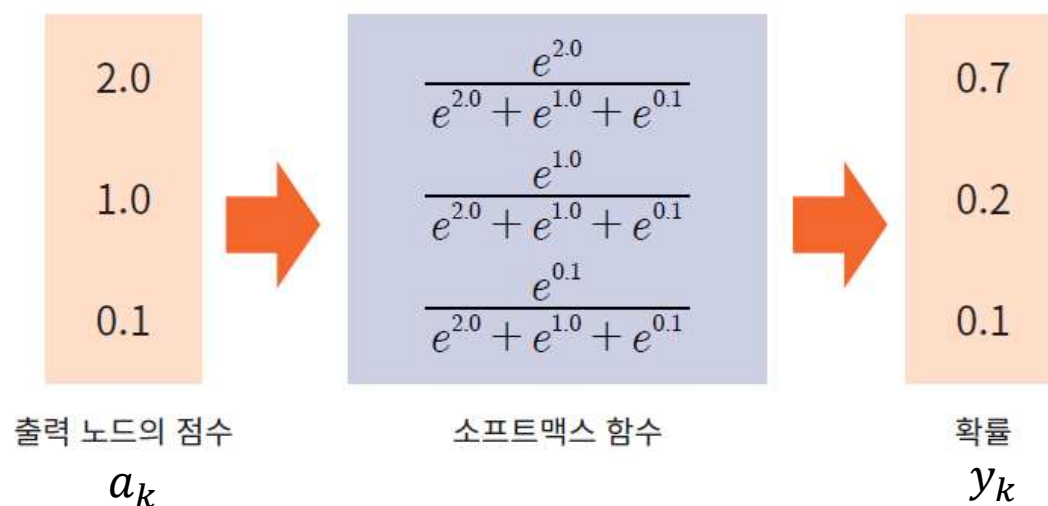
- 예를 들어서 목표값이 0.0이고 출력값이 10.0라고 하자. 차이는 무려 10.0이나 되지만 시그모이드 함수의 그래디언트는 거의 0이 된다.
- 반대로 목표값이 0.0이고 출력값이 1.0라고 하자. 차이는 1.0뿐이지만 그래디언트는 0.2 정도가 나오게 된다.
- 이것은 마치 시험성적이 안 좋은 학생한테 더 좋은 학점을 주는 격이다.



새로운 손실 함수

- 출력층의 노드에 소프트맥스 함수를 사용하고 손실 함수로 **교차 엔트로피**를 많이 사용한다
- 소프트맥스 활성화 함수 $y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$

교차 엔트로피를 쓰려면 출력을 소프트 맥스로 확률로 만들어줘야 한다.



큰 수는 더 커지고 작은 수는 더 작아집니다. 그리고 모든 수가 0.0부터 1.0 사이의 값이 됩니다.



그림 14-12 소프트맥스 함수의 계산 과정

교차 엔트로피 손실 함수

- 교차 엔트로피는 2개의 확률분포 간의 거리를 측정한 것이다.
- 교차 엔트로피는 2개의 확률 분포 p, q 에 대해서 다음과 같이 정의된다.

$$H(p, q) = - \sum_x p(x) \log q(x)$$

- 교차 엔트로피가 크면, 2개의 확률 분포가 많이 다른 것이다. 교차 엔트로피가 작으면 2개의 확률 분포가 거의 일치한다고 볼 수 있다.
- 목표값과 실제값이 일치하면 교차 엔트로피는 0이 된다.

교차 엔트로피 손실 함수

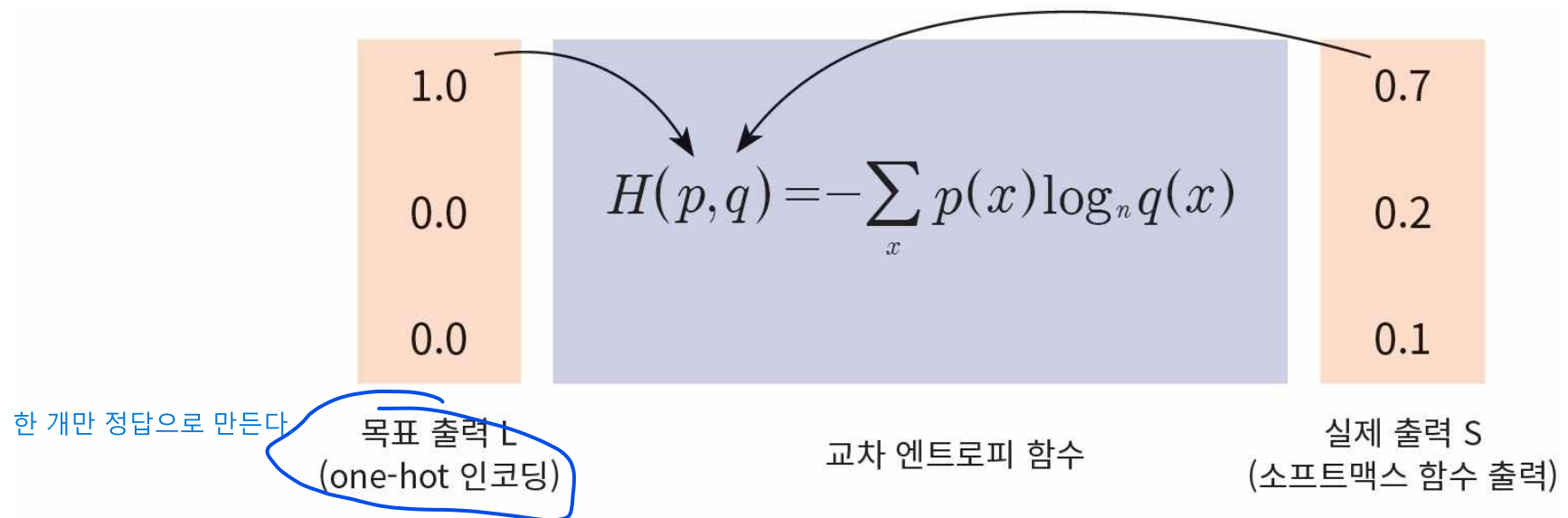


그림 14-9 교차 엔트로피 함수

$$\begin{aligned} H(p, q) &= -\sum_x p(x) \log_n q(x) \\ &= -(1.0 * \log 0.7 + 0.0 * \log 0.2 + 0.0 * \log 0.1) \\ &= 0.154901 \end{aligned}$$

완벽하게 일치한다면

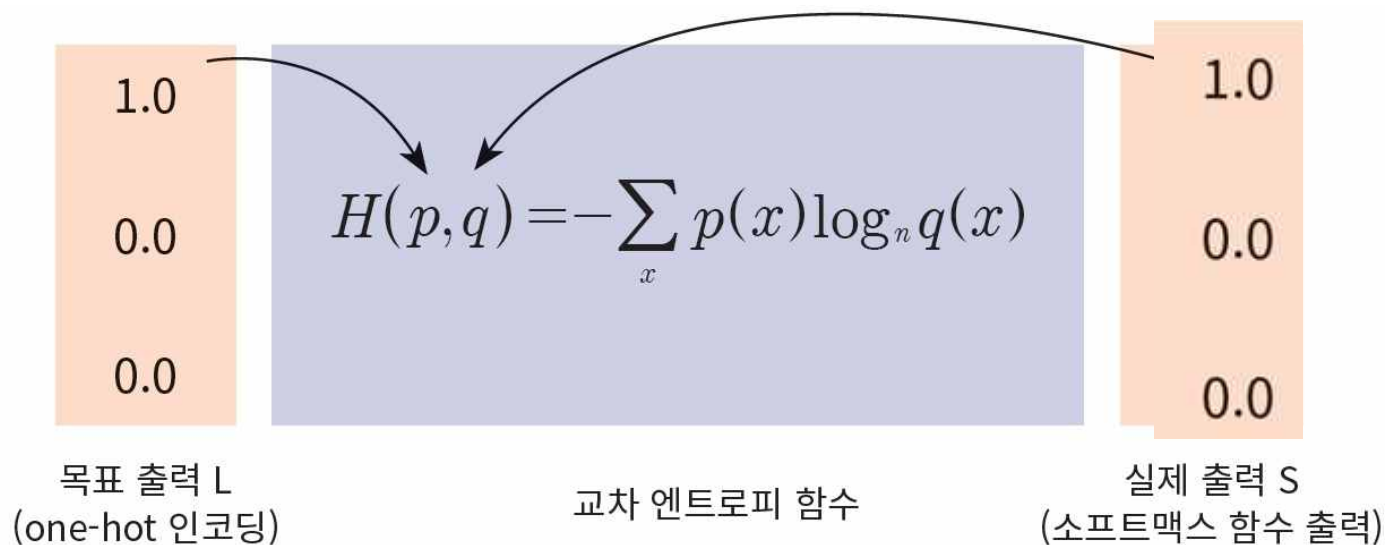


그림 14-9 교차 엔트로피 함수

$$\begin{aligned} H(p, q) &= -\sum_x p(x) \log_n q(x) \\ &= -(1.0 \times \log 1.0 + 0.0 \times \log 0.0 + 0.0 \times \log 0.0) \\ &= 0 \end{aligned}$$

Lab: 교차 엔트로피의 계산

입력 샘플	실제 출력			목표 출력		
샘플 #1	0.1	0.3	0.6	0	0	1
샘플 #2	0.2	0.6	0.2	0	1	0
샘플 #3	0.3	0.4	0.3	1	0	0

- 첫 번째 샘플에 대하여 교차 엔트로피를 계산해 보자. $-(\log(0.1) * 0 + \log(0.3) * 0 + \log(0.6) * 1) = -(0 + 0 - 0.51) = 0.51$ 이 된다.
- 두 번째 샘플의 교차 엔트로피는 $-(\log(0.2) * 0 + \log(0.6) * 1 + \log(0.2) * 0) = -(0 - 0.51 + 0) = 0.51$ 이다.
- 세 번째 샘플의 교차 엔트로피는 $-(\log(0.3) * 1 + \log(0.4) * 0 + \log(0.3) * 0) = -(-1.2 + 0 + 0) = 1.20$ 이다.
- 따라서 3개 샘플의 평균 교차 엔트로피 오류는 $(0.51 + 0.51 + 1.20) / 3 = 0.74$ 가 된다.

평균 제곱 오차 vs 교차 엔트로피

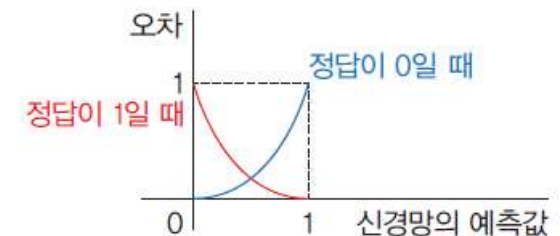
- 문제를 간단히 하기 위하여 출력 유닛이 하나인 경우만 생각하자

- 평균 제곱 오차: $E = (y - \hat{y}_i)^2$

- 이진 교차 엔트로피: $E = -[y \log_n(\hat{y}) + (1 - y) \log_n(1 - \hat{y})]$

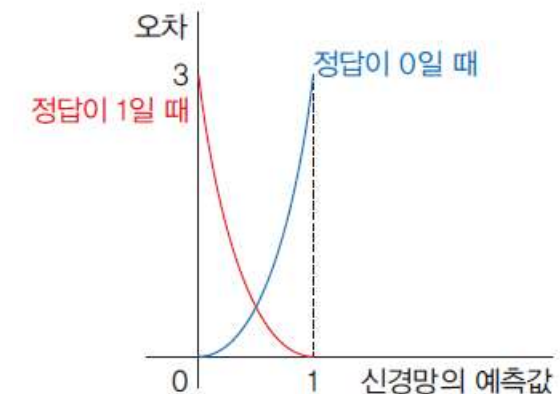
- 평균 제곱 오차(MSE): $E = \hat{y}_i^2$ (정답이 0일 때)

$$E = (1 - \hat{y}_i)^2 \text{ (정답이 1일 때)}$$



- 이진 교차 엔트로피(BCE) 오차: $E = -\log_n(1 - \hat{y})$ (정답이 0일 때)

$$E = -\log_n(\hat{y}) \text{ (정답이 1일 때)}$$



케라스에서 손실함수

- BinaryCrossentropy

- 이진 교차 엔트로피(BCE: Binary Cross Entropy)는 이진 분류 문제를 해결하는 데 사용되는 손실함수이다. 즉, 우리가 분류해야 하는 부류가 두 가지 뿐일 때 사용한다. 예를 들어 이미지를 ‘강아지’, ‘강아지아님’의 두 부류로 분류할 때 BinaryCrossentropy를 사용한다

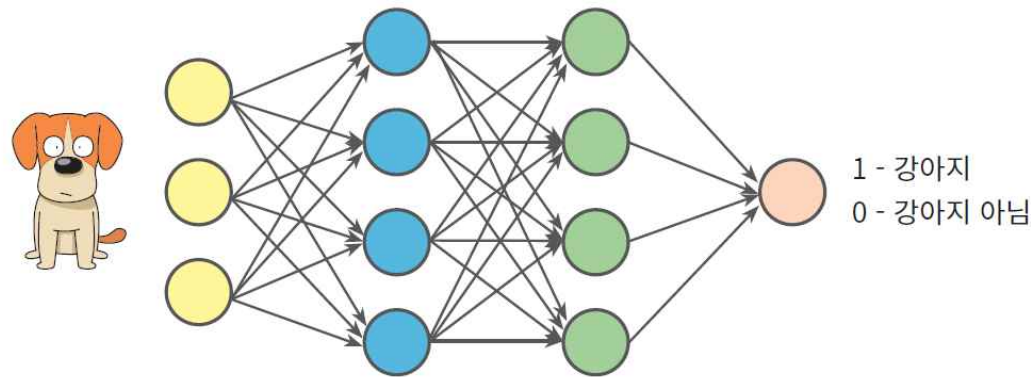


그림 14-14 BinaryCrossentropy 손실함수는 분류해야 하는 부류가 2개뿐일 때 사용한다.

$$BCE = -\frac{1}{n} \sum_{i=1}^n (y_i \log_n(\hat{y}_i) + (1 - y_i) \log_n(1 - \hat{y}_i))$$

실제 계산예

실제 레이블 y	1	0	0	1
예측 값 $\hat{y}$	0.8	0.3	0.5	0.9

$$\text{샘플 1: } BCE1 = -(1 \cdot \log(0.8) + (1-1) \cdot \log(1-0.8))$$

$$\text{샘플 2: } BCE2 = -(0 \cdot \log(0.3) + (1-0) \cdot \log(1-0.3))$$

$$\text{샘플 3: } BCE3 = -(0 \cdot \log(0.5) + (1-0) \cdot \log(1-0.5))$$

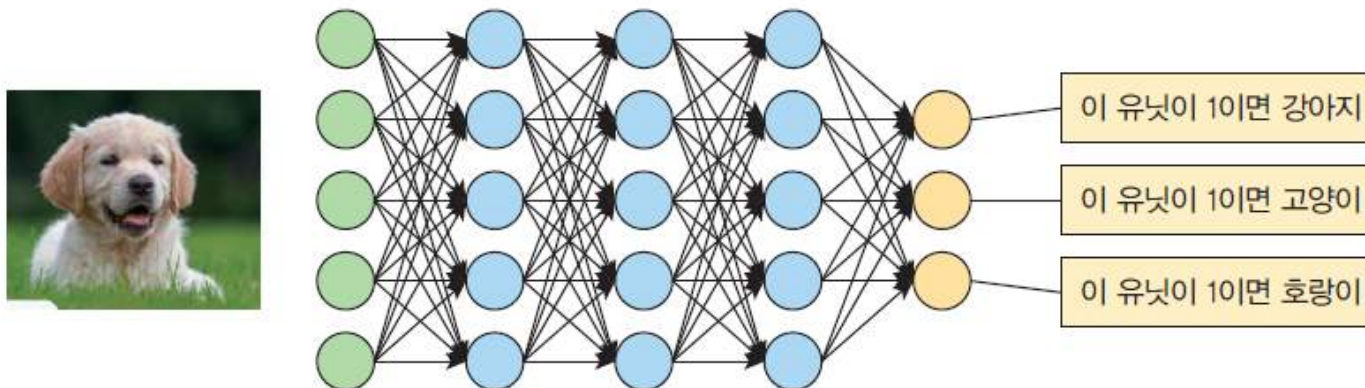
$$\text{샘플 4: } BCE4 = -(1 \cdot \log(0.9) + (1-1) \cdot \log(1-0.9))$$

$$BCE = \frac{BCE1 + BCE2 + BCE3 + BCE4}{4} \approx 0.345$$

```
y_true = [ [1], [0], [0], [1] ]  
y_pred = [[0.8], [0.3], [0.5], [0.9]]  
bce = tf.keras.losses.BinaryCrossentropy()  
print(bce(y_true, y_pred).numpy())
```

0.3445814|

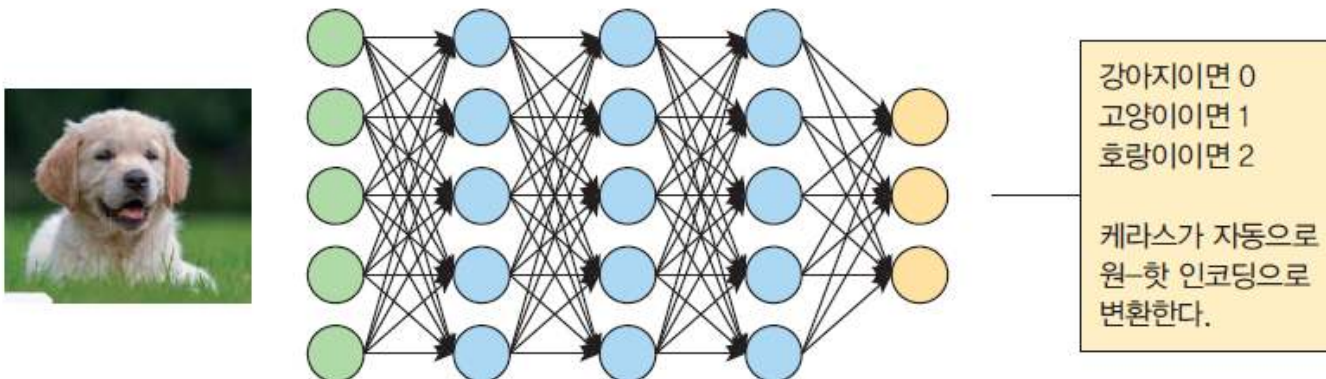
- CategoricalCrossentropy: 우리가 분류해야 할 부류가 두 개 이상이라면 (즉 다중 분류 문제라면) 을 사용
- 정답 레이블은 원-핫 인코딩으로 제공한다. 예를 들어서 입력 이미지를 “강아지(1,0, 0)”, “고양이(0, 1, 0)”, “호랑이(0, 0, 1)” 중의 하나로 분류



```
y_true = [[0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [1.0, 0.0, 0.0]] # 고양이, 호랑이, 강아지
y_pred = [[0.6, 0.3, 0.1], [0.3, 0.6, 0.1], [0.1, 0.7, 0.2]]
cce = tf.keras.losses.CategoricalCrossentropy ()
print(cce(y_true, y_pred).numpy ())
```

1.936381

- SparseCategoricalCrossentropy: 정답 레이블이 원-핫 인코딩이 아니고 정수로 주어지면 SparseCategoricalCrossentropy를 사용
- 예를 들어서 정답 레이블이 0(강아지), 1(고양이), 2(호랑이)로 주어지면 SparseCategoricalCrossentropy를 사용한다.



```
y_true = [1, 2, 0] # 고양이, 호랑이, 강아지
y_pred = [[0.6, 0.3, 0.1], [0.3, 0.6, 0.1], [0.1, 0.7, 0.2]]
scce = tf.keras.losses.SparseCategoricalCrossentropy()
print(scce(y_true, y_pred).numpy())
```

1.936381

케라스에서 손실함수

- MeanSquaredError

- 회귀 문제에서는 예측값과 실제값의 차이를 계산해야 한다.
MeanSquaredError 클래스는 예측과 실제값 사이의 평균 제곱 오차를 계산하는 데 사용할 수 있다.

```
y_true = [12, 20, 29, 60]  
y_pred = [14, 18, 27, 55]  
mse = tf.keras.losses.MeanSquaredError()  
print(mse(y_true, y_pred).numpy())
```

Q

- 교차 엔트로피란 무엇인가? 교차 엔트로피를 손실 함수로 사용하려면 목표 출력과 실제 출력을 어떻게 만들어야 하는가?

답은 시아

100개중에서 하나만 일부로 만들어야 한
분류하는 클래스 개수만큼 만들어야 한다

soft max 함수로 실제 출력을 바꿔주어야 한다.
총합이 1이 되는 확률로 바꾸어 준다.

04 가중치 초기화 문제

- 출력층에서 계산된 오차는 가중치의 값에 비례하여서 역전파된다 (가중치가 곱해져서 역전파된다).
- 따라서 모든 가중치가 0이면 오차 역전파가 제대로 되지 않는다.

가중치를 0으로 해버리면 0을 곱하니까 제대로 안된다.
초기 값을 0으로 설정해버리면 안된다.

$$\delta_j = \begin{cases} (out_j - t_j) f'(net_j) & j \text{가 출력층 노드이면} \\ (\sum_k W_{jk} \delta_k) f'(net_j) & j \text{가 은닉층 노드이면} \end{cases}$$

k는 내 뒤에 있는 값을 가져오는 건다.

그림 14-17 신경망에서는 초기 가중치값도 지대한 영향을 끼친다.

최대 값 0.25

0.25×0.25

가중치가 0이라면 델타가 전달되지 않아요!

렐루면 $y=x$ 이니까 미분한 값이 1이라서 1을 곱해진다.
시그모이드랑 다르게 1을 곱해서 사라지지 않는다.

가중치 초기화 문제

- 가중치 w_1, w_2, w_3, w_4 와 가중치 w_5, w_6, w_7, w_8 이 모두 같은 값(예를 들어서 0.2)으로 설정되어 있다면, 노드 E와 노드 G는 완벽하게 동일한 일을 하게 된다. 이것을 방지하는 것을 “균형 깨뜨리기(breaking the symmetry)”라고 부르고 있다.

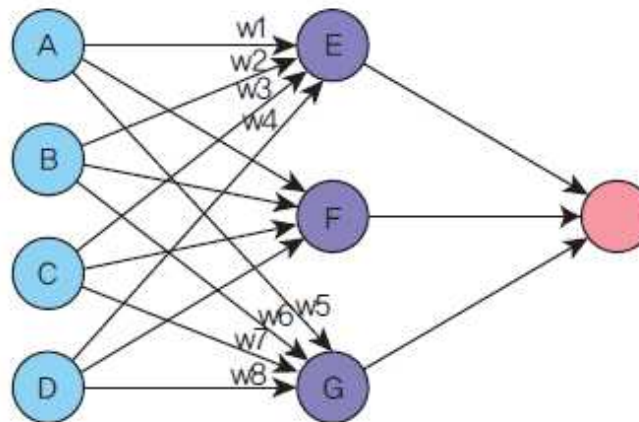
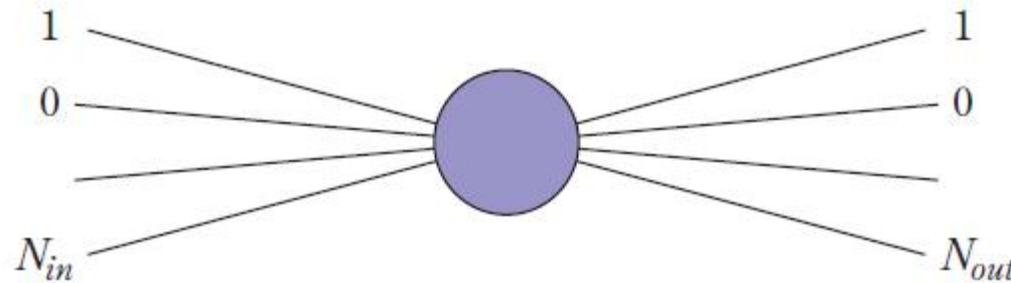


그림 8-10 가중치가 동일할 때의 문제점

난수 가중치 초기값

- 이상과 같은 이유로 가중치의 초기값은 난수로 결정되어야 한다.
- 또 반대로 너무 큰 가중치는 그래디언트 폭발(exploding gradients)을 일으킨다고 한다. 이 경우에는 학습이 수렴하지 않고 발산하게 된다
- **Xavier**의 방법 (활성화 함수가 sigmoid나 tanh일때 사용): 다음과 같은 분산을 가지는 정규 분포(가우시안 분포)에서 난수를 추출하는 것을 제안하였다.
 N_{in} : 앞 계층의 노드 개수

자비에 초기화 방법



$$var(w_i) = \frac{2}{(N_{in} + N_{out})}$$

출력 노드 개수도 더해진다

가중치 초기화 방법

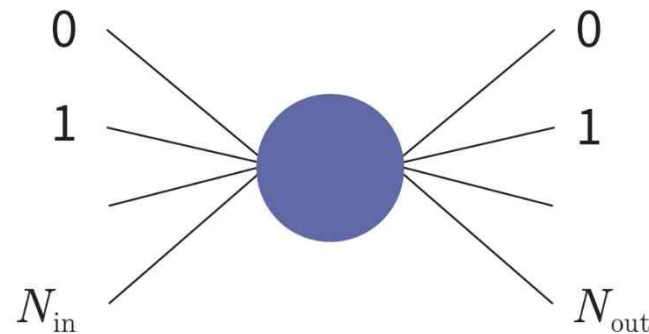
- He Kaiming은 다음과 같은 분산을 가지는 정규 분포(가우시안 분포)에서 난수를 추출하는 것을 제안하였다. N_{in} : 앞 계층의 노드 개수

입력 노드 개수

분산 $\text{Var}(w_i) = \frac{2}{N_{in}}$: He 초기화, ReLU

분산이 저거인 정규 분포에서 뽑으면 줄

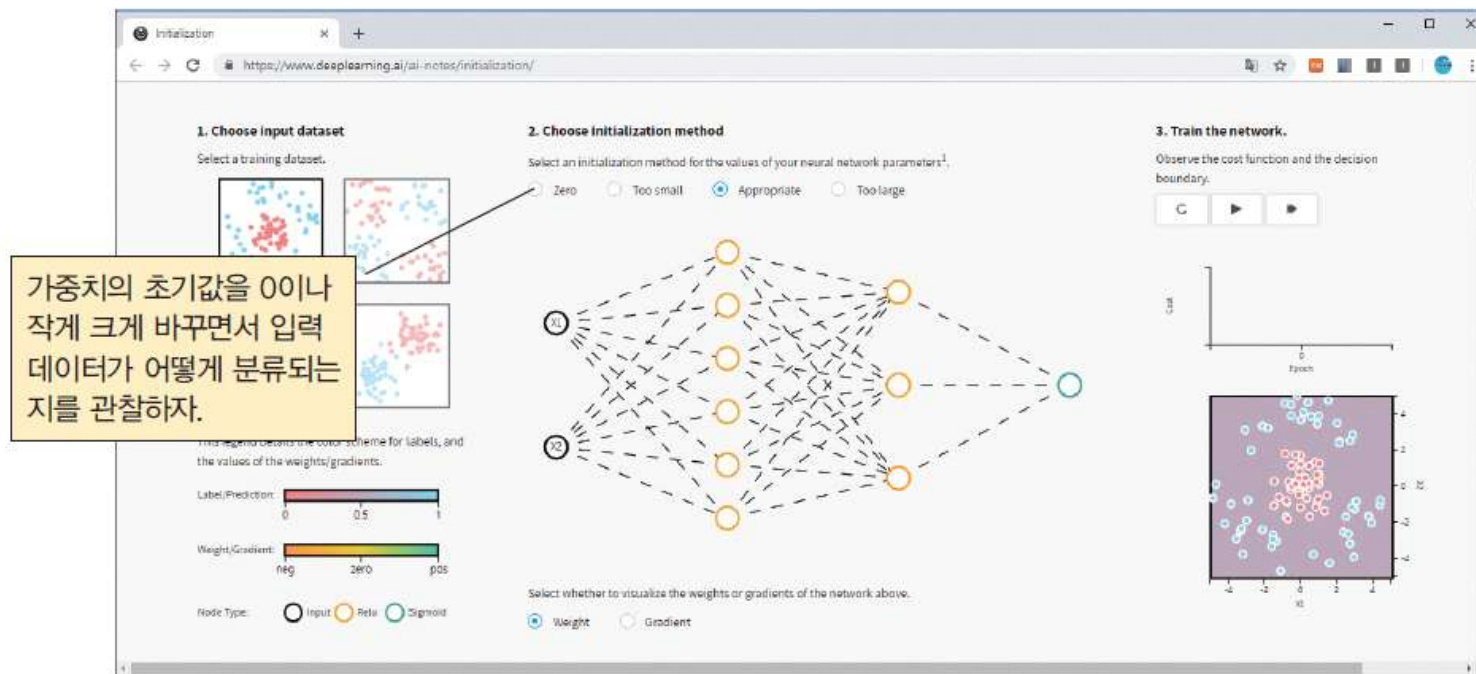
요즘 가장 많이 쓰
해 초기화 방법.



렐루가 활성화 함수에 대한

Lab: 가중치 초기화 실험

- <https://www.deeplearning.ai/ai-notes/initialization/>에 가보면 여러분들은 다양한 초기값을 가지고 오차가 얼마나 빨리 줄어드는지를 볼 수 있다.



Q

- DNN에서 초기 가중치는 어떻게 설정하는 것이 좋은가? 최악의 가중치는 무엇인가?

0으로 하는 게 최악일 듯. ($2/\text{입력 노드}$)?

활성화가 렐루 일 때는 해 초기화
 $2/\text{입력노드 개수}$

활성화 함수가 시그모이드나 하이퍼 탄젠트 일때는 자비에 초기화
 $2/(\text{입력노드} + \text{출력노드})$

05 미니 배치

- 확률적 경사 하강법(**Stochastic Gradient Descent: SGD**) : 확률적으로 랜덤하게 선택되는 일부의 샘플이 주어지면 오차를 계산하여서 바로 가중치를 변경하는 방법이다.
- 배치 학습(batch learning): 모든 샘플을 모두 보여준 후에 개별 샘플의 그래디언트를 전부 더해서 이것을 바탕으로 모델의 가중치를 변경하는 방법이다.

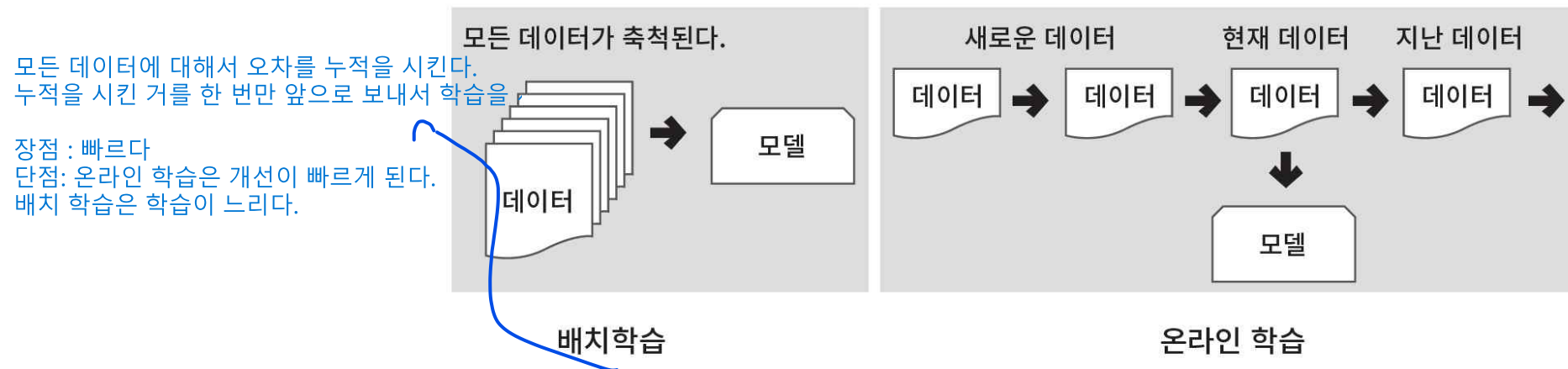


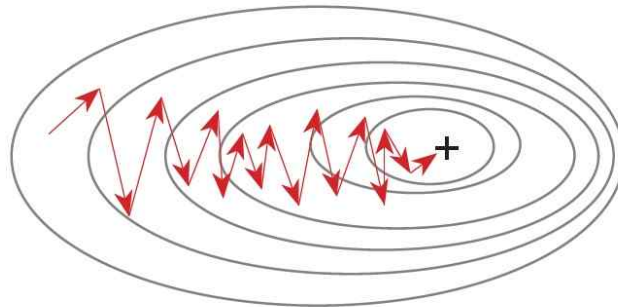
그림 14-11 배치 학습과 온라인 학습

배치학습은 데이터가 대규모이면 메모리가 감당이 안됨
요즘은 보통 미니 배치를 한다.

미니 배치

- 온라인 학습과 배치 학습의 중간에 있는 방법이 미니 배치(mini batch)이다. *batch*
- 이 방법에서는 훈련 데이터를 작은 배치들로 분리시켜서 하나의 배치가 끝날 때마다 학습을 수행하는 방법이다.
- $1 < \text{size}(\text{미니 배치}) < \text{size}(\text{훈련 데이터})$

확률적 경사 하강법(SGD)



미니 배치 경사 하강법

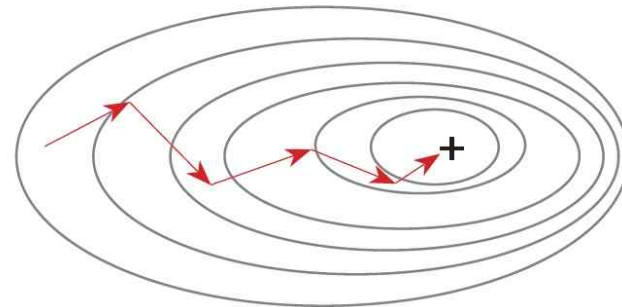


그림 14-12 온라인 학습과 미니 배치

데이터를 몇 개 단위로 나누어서 학습을 시키는 것이 미니 배치이다.

온라인 학습은 노이즈가 섞일 수 있어서 흔들리는데, 미니 배치 한 노이즈가 영향력이 줄어드니까 덜 흔들리는 경향이 있다.

Q

- 훈련 데이터를 작은 배치들로 분리시켜서 하나의 배치가 끝날 때마다 학습을 수행하는 방법을 무엇이라고 하는가?

미니 배치

06 데이터 정규화

- Z-score 정규화 (StandardScaler)

- $x_j' = \frac{x_j - \mu_j}{\sigma_j}$ 특징에서 표준편차를 뺀다. 특징의 표준편차로 현재 값에서 평균을 뺀 것을 알면 정규화가 된다
- 결과 범위: 평균 0, 표준편차 1, [-3, 3] 사이로 99.7% 변환됨

1. 일반 테이블 데이터 (MLP)

```
from sklearn.preprocessing import StandardScaler scaler =  
StandardScaler() # Z-score
```

```
X_scaled = scaler.fit_transform(X)      —> Z-score 정규화가 적용이 됨
```

2. 이미지 데이터 (CNN 등)

```
import torchvision.transforms as T
```

```
transform = T.Compose([  
    T.ToTensor(), # [0~255] → [0.0~1.0]      # RGB 각 채널별로  
    T.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet 평균빼줌  
                 std=[0.229, 0.224, 0.225]) # ImageNet 표준편차  
    로 나눠줌 ])
```

- 이미지 데이터 정규화

데이터에 픽처가 여러 가지가 있다 픽처가 2가지인 경우

- 채널별 Z-score 정규화 (ImageNet 정규화)
- Min-Max 정규화 (옛날 방식, 지금은 금지)

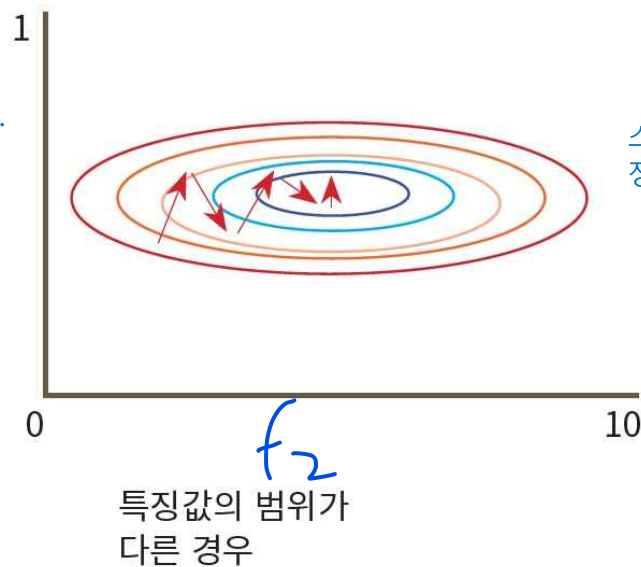
- $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$, 범위: $[0, 1]$ 민 맥스 정규화 옛날 방식이다.

특징 1 : 0~1
특징 2: 0~10

이런 경우 큰 숫자 영향을 많이 받는다.

학습이 잘 안된다.
그래서 그걸 스케일을 맞춘다

이것이 정규화



스케일 맞추는
정규화

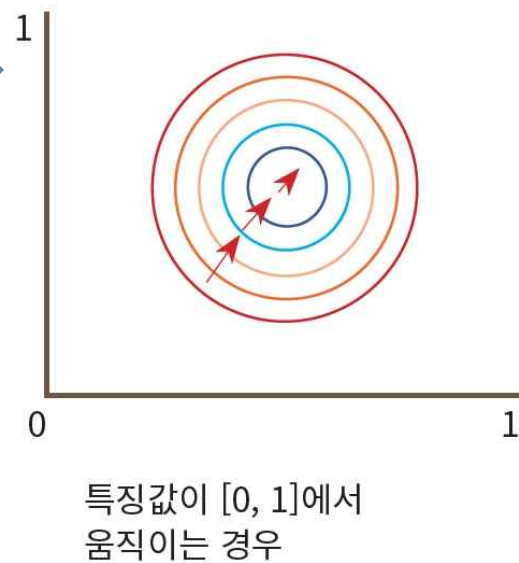
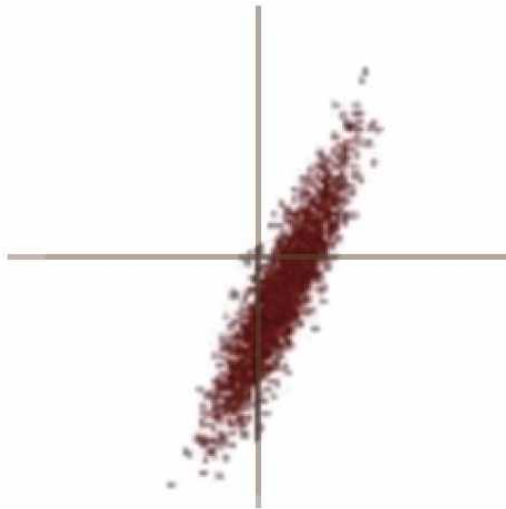


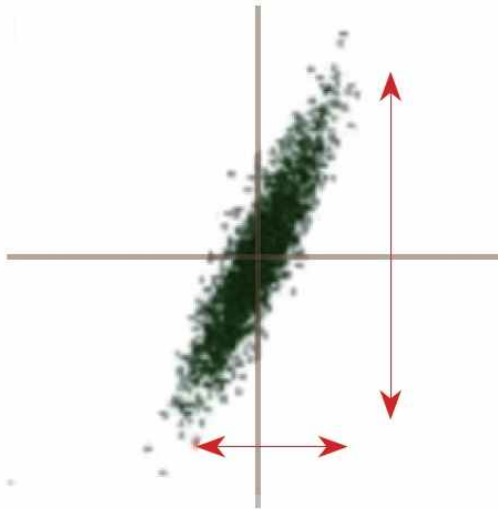
그림 14-13 데이터 정규화의 영향

데이터 정규화

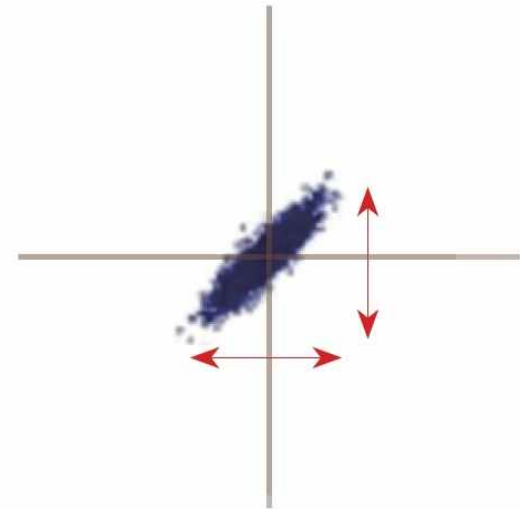
원래의 데이터



원점이 데이터의 중심이 되도록 함



정규화된 데이터



Q

- **DNN**에서 가장 많이 사용하는 입력값 변환 방법은 무엇인가? 이것을 무엇이라고 부르는가?

데이터 정규화

07 데이터 인코딩 방법

- 입력 데이터 중에는 “male”, “female”과 같이 카테고리를 가지는 데이터들이 아주 많다. -> 숫자로 바꾸어야 한다.

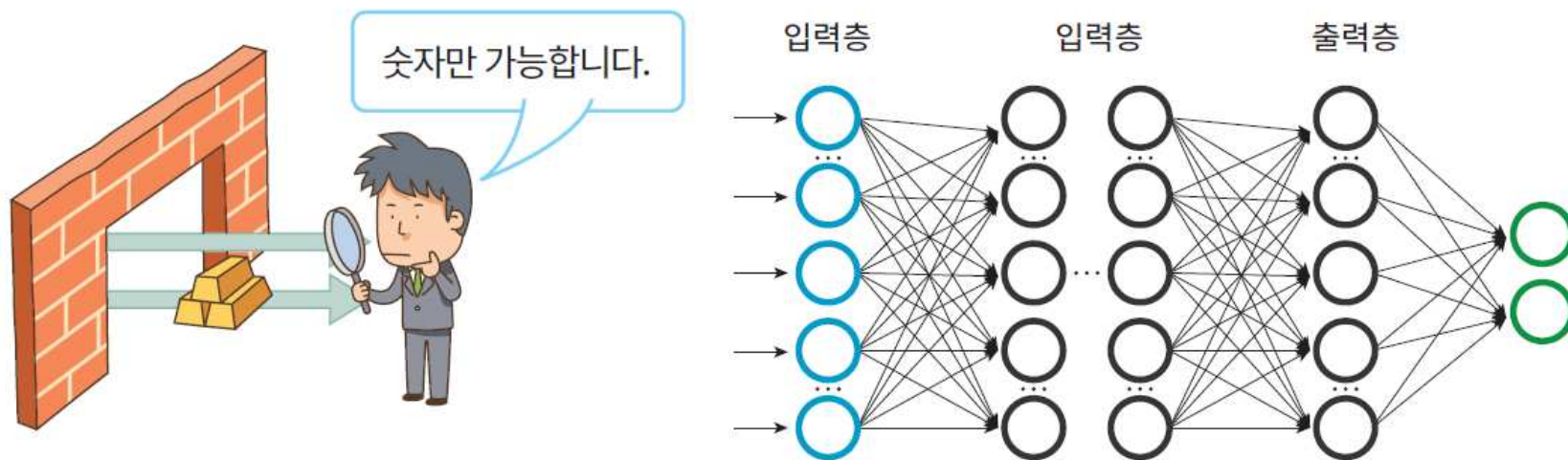


그림 14-22 데이터 인코딩 기법의 예

입력이 숫자가 되어야 한다. 근데 훈련 시키고 싶은 데이터가 숫자가 아닌 경우가 있다 그런 것을 바꾸는 경우

간단한 경우

```
for ix in train.index:  
    if train.loc[ix, 'Sex']=="male":  
        train.loc[ix, 'Sex']=1  
    else:  
        train.loc[ix, 'Sex']=0
```

일반적인 범주형 데이터 변환 방법

- 범주형 변수를 인코딩하는 3가지 방법
 - 정수 인코딩(Integer Encoding): 각 레이블이 정수로 매핑되는 경우이다.
 - 원-핫 인코딩(One Hot Encoding): 각 레이블이 이진 벡터에 매핑되는 경우이다.
 - 임베딩(Embedding): 범주의 분산된 표현이 학습되는 경우이다.

원-핫 인코딩과 비교:

- 원-핫 인코딩: 서로 독립된 값으로 표현 (범주 간 관계 없음).
- 임베딩: 벡터 간 거리와 방향이 의미를 가짐 (예: 유사도, 관계성).

범주	원-핫 인코딩	임베딩 (학습된 벡터)
apple	[1, 0, 0]	[0.2, -0.8, 0.5]
banana	[0, 1, 0]	[0.1, -0.7, 0.6]
orange	[0, 0, 1]	[0.3, -0.6, 0.4]

1의 위치로 구분

요즘 쓰는 방법, 해당 값을 실수 벡터로 바꾸는 것.

정수 인코딩

- sklearn 라이브러리가 제공하는 Label Encoder 클래스를 사용

```
import numpy as np
X = np.array([[ 'Korea', 44, 7200],
              [ 'Japan', 27, 4800],
              [ 'China', 30, 6100]])
```

이것만 바꾸는 예제

```
from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
X[:, 0] = labelencoder.fit_transform(X[:, 0])
print(X)
```

```
[[ '2' '44' '7200']
 [ '1' '27' '4800']
 [ '0' '30' '6100']]
```

원-핫 인코딩

- 원-핫 인코딩은 단 하나의 값만 1이고 나머지는 모두 0인 인코딩을 의미한다.

Country	Age	Salary
Korea	38	7200
Japan	27	4800
China	30	3100



Korea	Japan	China	Age	Salary
1	0	0	38	7200
0	1	0	27	4800
0	0	1	30	3100

```
import numpy as np
X = np.array(['Korea', 44, 7200],
              ['Japan', 27, 4800],
              ['China', 30, 6100])

from sklearn.preprocessing import OneHotEncoder
onehotencoder = OneHotEncoder()

# 원하는 열을 뽑아서 2차원 배열로 만들어서 전달하여야 한다.
XX = onehotencoder.fit_transform(X[:,0]).toarray()
print(XX)

X = np.delete(X, [0], axis=1)      # 0번째 열 삭제
X = np.concatenate((XX, X), axis = 1) # X와 XX를 붙인다.
print(X)
```



```
[[0. 0. 1.]  
 [0. 1. 0.]  
 [1. 0. 0.]]  
[['0.0' '0.0' '1.0' '44' '7200']  
 ['0.0' '1.0' '0.0' '27' '4800']  
 ['1.0' '0.0' '0.0' '30' '6100']]
```

원-핫 인코딩(케라스 사용)

- 원-핫 인코딩은 케라스의 `to_categorical()`을 호출해서도 만들 수 있다.

```
class_vector = [2, 6, 6, 1]
```

```
from tensorflow.keras.utils import to_categorical  
output = to_categorical(class_vector, num_classes = 7, dtype = "int32")  
print(output)
```

```
[[0 0 1 0 0 0 0]  
 [0 0 0 0 0 0 1]  
 [0 0 0 0 0 0 1]  
 [0 1 0 0 0 0 0]]
```

- Embedding
 - 딥러닝이 이해할 수 있도록, 단어·아이템·사용자 같은 “카테고리”를 실수 벡터로 바꿔주는 것
 - 쉽게 말하면 “컴퓨터가 ‘고양이’와 ‘강아지’가 비슷하다는 걸 숫자로 느끼게 해주는 마법”
- 2025년 딥러닝에서 데이터 인코딩
 - = 거의 100% "정수로 바꾸고 → Embedding 층에 넣는 것"
 - 텍스트는 정수로 바꾸고(HuggingFace 토큰나이저) → Embedding,
 - 카테고리는 Label Encoding → Embedding
 - One-Hot은 입력에는 절대 금지! 원핫은 입력에선 퇴출됐지만, 정답 라벨에선 여전히 왕입니다!

원핫은 출력에서 쓰인다, 입력에서는 x 교차 엔트로피 할 때 쓰인다.

● Embedding 비교

입력 예시	기존 방식One-Hot	Embedding 방식	차원 수 예시
단어: “고양이 ”	[0, 0, 1, 0, 0, 0, 0...] 10만 차원	[0.12, -0.85, 0.34, 0.67, -0.23] 300차원	768차원 (BERT) 300~1024차원
사용자 ID: 12345	100만명 ->100만 차원 0과 1	[0.41, -0.12, 0.89, ..., 0.05] 64차원	32~256차원
영화 제목: “인셉션 ”	5만개 영화 ->5만 차원	[0.55, 0.33, ...] 384차원 실수 벡터	384~768차원

● Embedding 사용 이유

이유	설명	예시 비유
100만 명이 있다면 100만 차원이 된다 , 근데 임베딩을 사용하면 64차원이		
1.차원이 작고 밀집	One-Hot은 10만 단어 -> 10만 차원 Embedding은 256~1024차원만	지하철 노선도 VS 실제 지도
2.의미가 담겨서 의미.유사도가 반영됨	“고양이”와 “강아지 ” 는 벡터가 서로 가깝게 학습됨. “고양이 ” 와 “자동차 ” 는 멀리 떨어짐	왕-남자+여자 = 여왕
3. 학습이 가능	처음엔 랜덤 → 훈련하면서 점점 똑똑한 벡터로 변함	모델이 자동으로 의미를 배움

08 학습률

- 학습률이란 한 번에 가중치를 얼마나 변경할 것인가를 나타낸다. 학습률이 너무 크게 설정되면 오버슈팅이 일어난다.

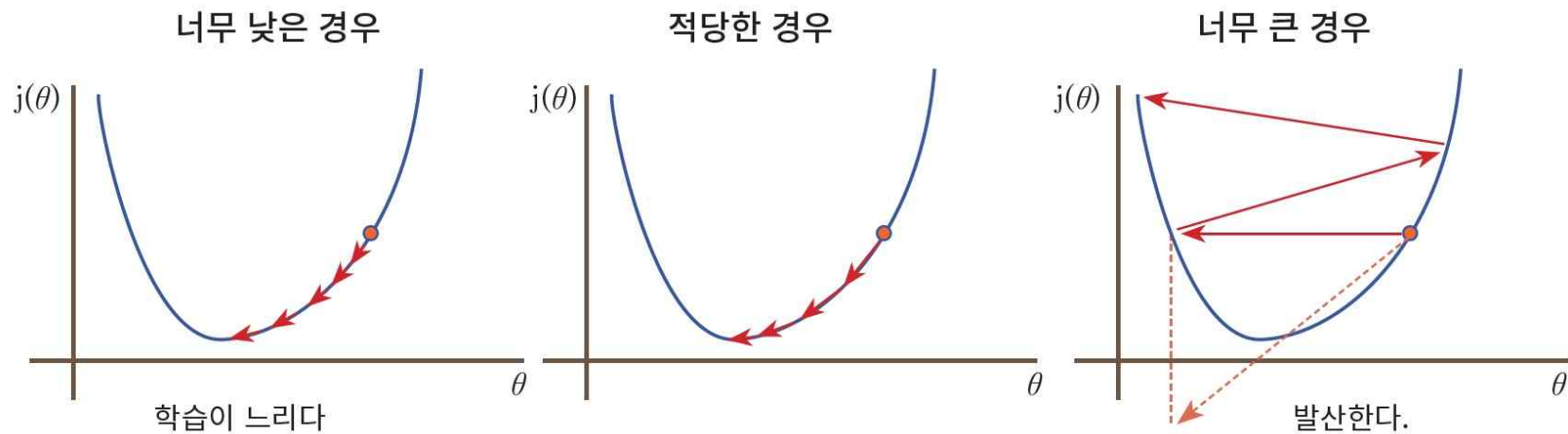


그림 14-15 학습률 과소 적합

과잉 적합

모멘텀

- 모멘텀(momentum)은 학습 속도를 가속시킬 목적으로 사용한다.

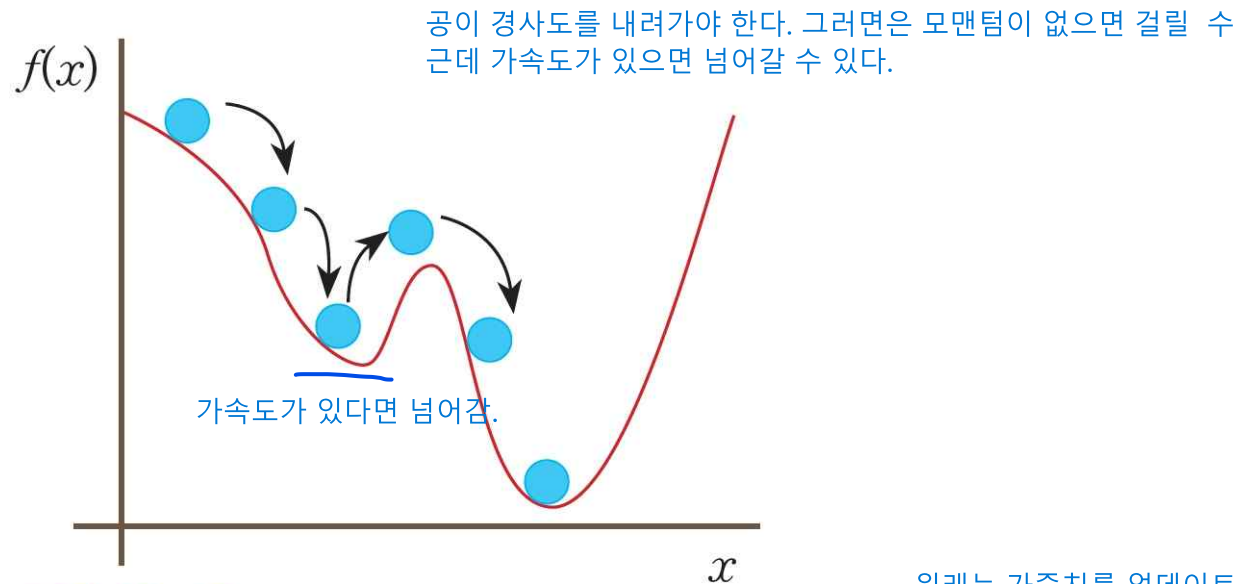


그림 14-16 모멘텀

원래는 가중치를 업데이트할 때 에러의 미분값 그래디언트를 빼준다

$$w_{ij}(t+1) = w_{ij}(t) - \eta \frac{\partial E}{\partial w_{ij}} + \underbrace{\text{momentum} * w_{ij}(t)}$$

기존의 가중치를 일정부분 더해주는 것이다
기존에 가던 관성이 추가되는 효과가 생긴

09 과잉 적합

- 과잉 적합(**over fitting**)은 지나치게 학습 데이터에 특화돼 실제 적용 시 좋지 못한 결과가 나오는 것을 말한다.

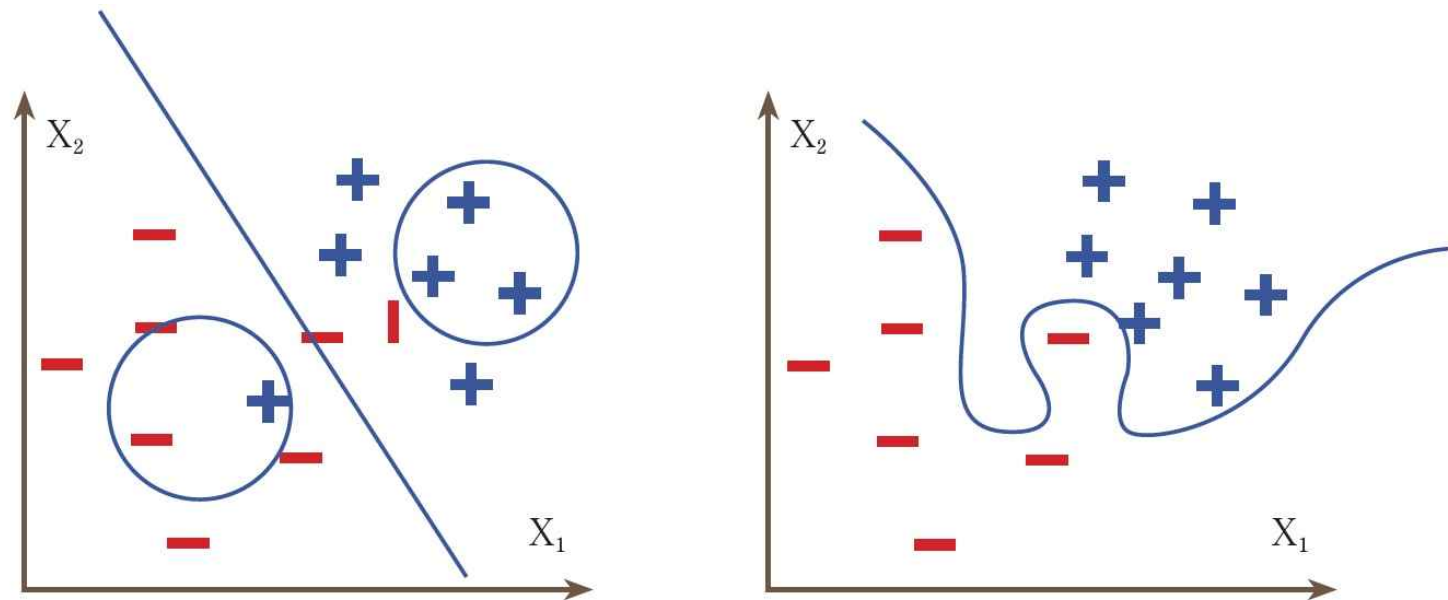


그림 14-17 과잉 적합의 예

과잉적합

- 과잉 적합은 어떻게 알 수 있을까? 검증 데이터셋으로 알 수 있다. 신경망을 훈련하면서 신경망이 보지 못한 검증 데이터셋으로 검증해보는 것이다. 훈련이 계속되어도 검증 데이터의 손실값이 감소하지 않는다면 과잉 적합이 발생한 것이다.

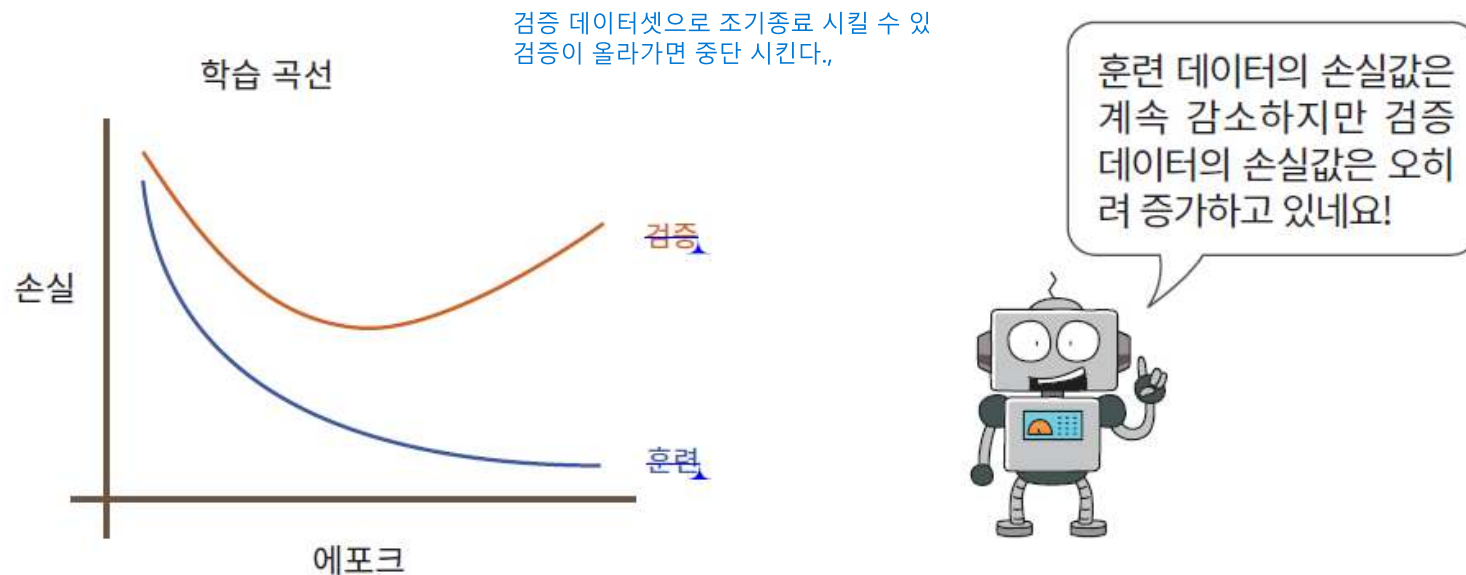
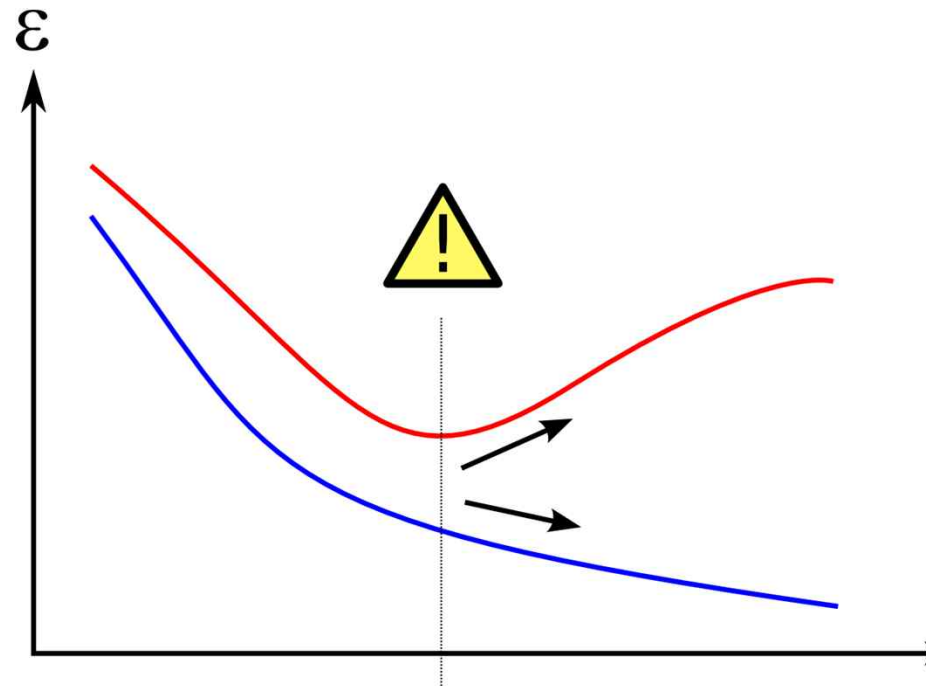


그림 14-26 검증 데이터에 대하여 성능이 나빠지고 있다면 과잉 적합이 발생한 것이다.

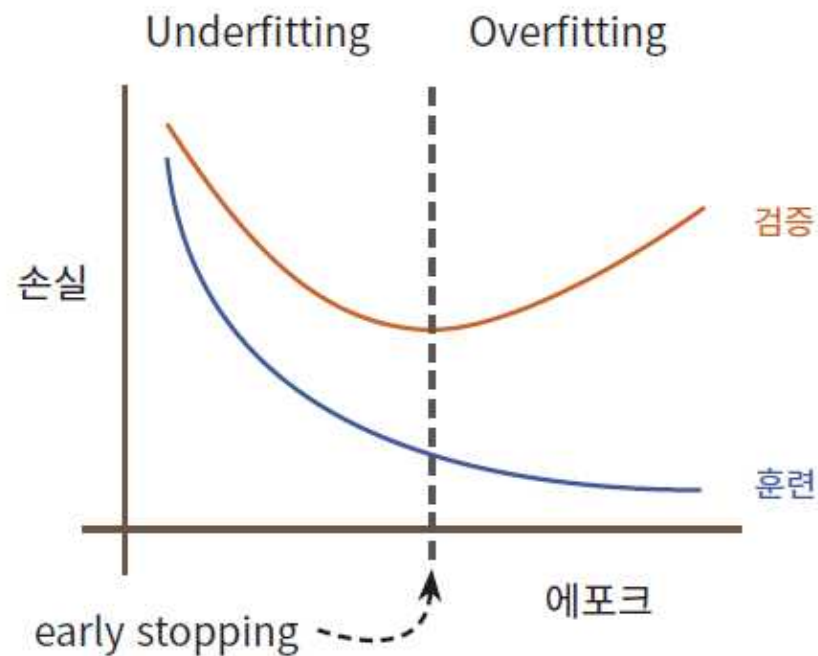
과잉 적합 처리 방법

- **조기 종료**: 검증 손실이 증가하면 훈련을 조기에 종료한다.
- 가중치 규제 방법: 가중치의 절대값을 제한한다.
- 데이터 증강 방법: 데이터를 많이 만든다.
- 드롭 아웃 방법: 몇 개의 뉴런을 쉬게 한다.



조기 종료

- 검증 손실이 더 이상 감소하지 않는 것처럼 보일 때마다 훈련을 중단할 수 있다. 이런 식으로 훈련을 중단하는 것을 조기 종료(Early Stopping)라고 한다



가중치 규제

- 연구자들은 복잡한 신경망 모델에서 가중치의 값이 너무 크면, 판단 경계선이 복잡해지고 과잉 적합이 일어난다는 사실을 발견하였다.
- 따라서 과잉 적합을 완화시키는 방법은 가중치가 작은 값을 가지도록 신경망의 복잡도에 제약을 가하는 것이다

- L1 규제는 가중치의 절대값에 비례하는 비용을 손실 함수에 추가하는 방법이다.

가중치가 커지는 것을 막는다.
가중치가 커지면 발산을 해서 학습이 안된
가중치를 줄이는 것이 이 방법.

오차에다가 가중치의 절대값을 일정부분 더해준다.

$$Loss = \underset{\text{오차}}{Cost} + \lambda \sum |W|$$

미분하면 가중치의 부호만 남게 된
부호가 일정부분 깨지는 것이다

- L2 규제는 가중치의 제곱에 비례하는 비용을 손실 함수에 추가하는 방법이다. 이것을 가중치 감소(weight decay)라고도 부른다.

$$Loss = Cost + \lambda \sum W^2$$

가중치의 제곱을 일정부분 더해준다

L1 Regularization

- L1 Regularization을 사용해 새롭게 정의된 **cost function**

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

- L1 Regularization의 개념에서 가장 중요한 것은 **cost function**에 가중치의 절대값을 더해준다는 것이기 때문에
- 기존의 **cost function**에 가중치(**W**)의 크기가 포함되면서,
- 학습의 방향이 단순히 **cost function**의 값이 작아지는 방향으로만 진행되는 것이 아니라,
- 가중치(**W**) 또한 작아지는 방향으로 학습이 진행됨
- 이때 λ 는 상수로 0에 가까울 수록 정규화의 효과는 없어짐
- $w \rightarrow$ 새롭게 정의된 **cost function**을 **w**에 대해 편미분한 결과

$$w \rightarrow w - \frac{\eta \lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w}$$

- **w**의 크기와 상관없이 **w**의 부호에 따라 상수값을 빼주는 방식

L2 Regularization

- L2 Regularization을 사용해 새롭게 정의된 cost function

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

- 기존의 cost function 에 가중치의 제곱을 더함으로써
- L1 Regularization 과 마찬가지로 가중치가 너무 크지 않은 방향으로 학습됨
- 새롭게 정의된 cost function을 w 에 대해 편미분한 결과
 - w 에 $(1 - \eta \lambda/n)$ 을 곱함으로써 w 값이 작아지는 방향으로 진행
 - 이를 Weight decay라고 함

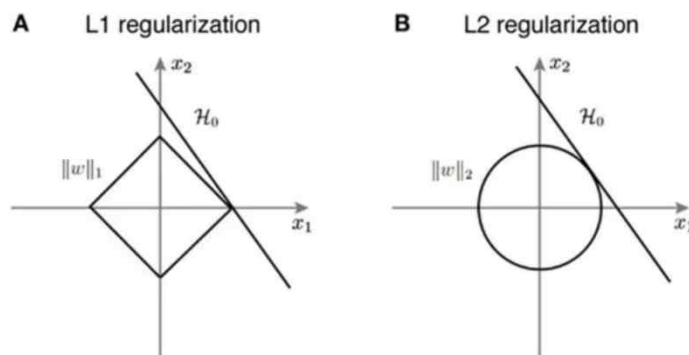
$$w \rightarrow w - \eta \frac{\partial C_0}{\partial w} - \frac{\eta \lambda}{n} w$$

크기가 들어가기 때문에 L2규제가 효과가 더 좋다.

$$= \left(1 - \frac{\eta \lambda}{n}\right) w - \eta \frac{\partial C_0}{\partial w}$$

L1 Regularization, L2 Regularization 의 차이와 선택 기준

- L1 Regularization은 가중치 업데이트 시, 가중치의 크기에 상관 없이 상수값을 빼면서 진행됨
 - 때문에 작은 가중치들은 거의 0으로 수렴 되어, 몇개의 중요한 가중치들만 남게됨
 - 그러므로 몇 개의 의미 있는 값을 꼬집어내고 싶은 **sparse model** 같은 경우에는 L1 Regularization이 효과적임
 - 다만, L1 Regularization은 아래 그림처럼 미분 불가능한 점이 있기 때문에 **Gradient-base learning** 에는 주의가 필요함



- L2 Regularization은 가중치 업데이트 시, 가중치의 크기가 직접적인 영향을 미침
 - 따라서, L2는 L1 보다 가중치 규제에 좀 더 효과적임

데이터 증강 (data augmentation)

- 소량의 훈련 데이터에서 많은 훈련 데이터를 뽑아내는 방법이다.

데이터가 적으면 오버피팅이 일어나기가 쉽다. 데이터를 억지로 늘려버린다.

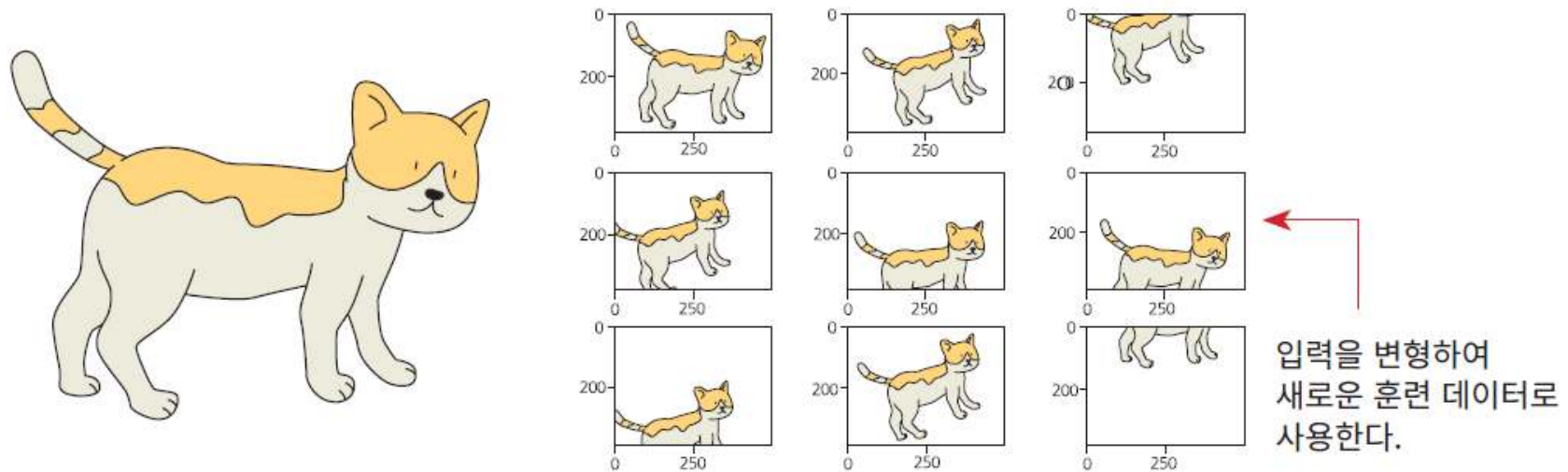


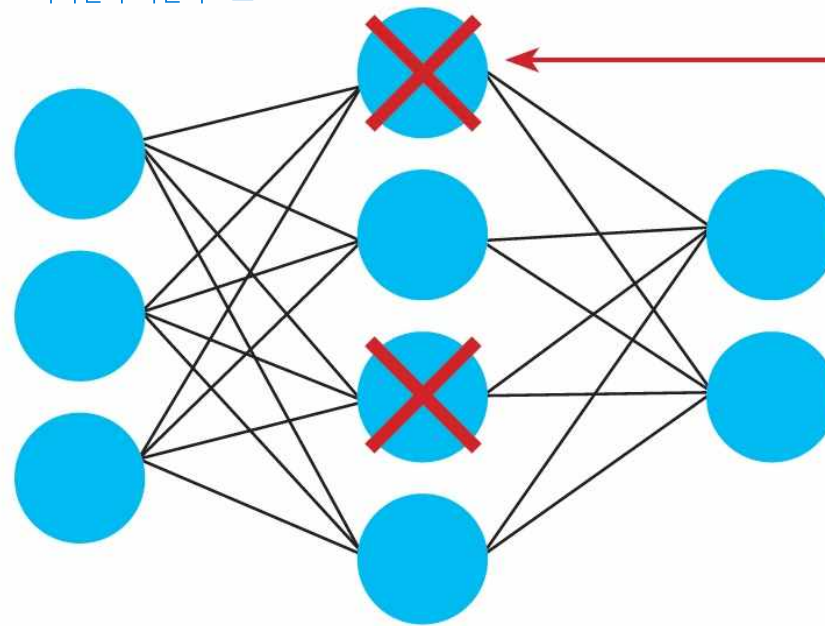
그림 14-27 데이터 증강 방법

드롭아웃

- 드롭아웃은 몇 개의 노드들을 학습 과정에서 랜덤하게 제외하는 것이다.

어느 층의 노드를 20퍼센트 꺼버린다 이런식으로

꺼놓지 않으면 특정 노드는 특정일만 한
꺼버리면 하던일이 다른 애 한테 넘어가
골고루 일 능력이 상승한다.



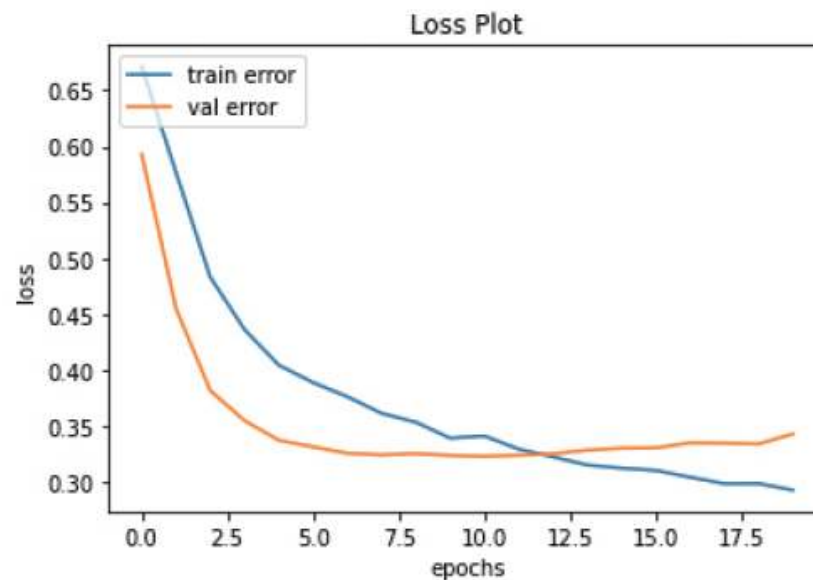
드롭아웃은 학습의
도중에 몇 개의 노드를
쉬게 하는 것이다.

그림 14-19 드롭아웃

신경망 모델 구축

```
model = tf.keras.Sequential()  
model.add(tf.keras.layers.Dense(16, activation='relu', input_shape=(10000,)))  
model.add(tf.keras.layers.Dropout(0.5))  
model.add(tf.keras.layers.Dense(16, activation='relu'))  
model.add(tf.keras.layers.Dropout(0.5))  
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
```

앞에 층에서 전달되는 거의 50프로를 건너뛴다?



드롭 아웃의 효과

- 랜덤하게 노드들을 쉬게 해서 학습을 시키면 동일한 특징을 검출하는 전문가가 많이 생기게 되는 것이다.

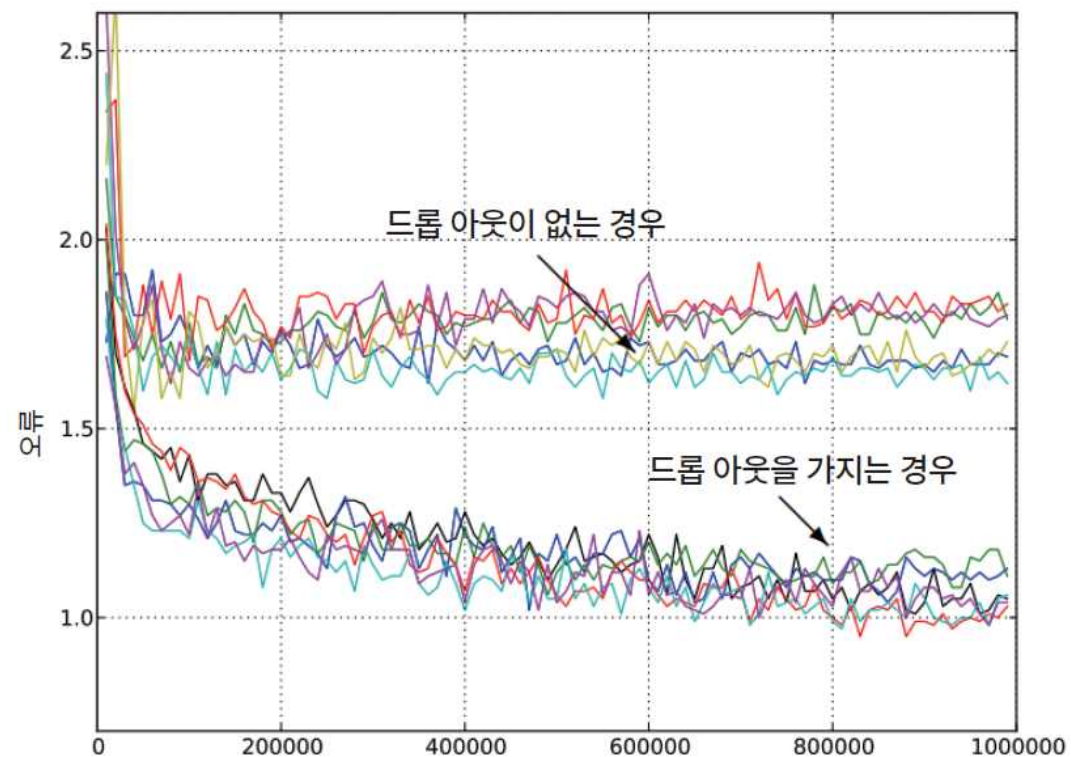
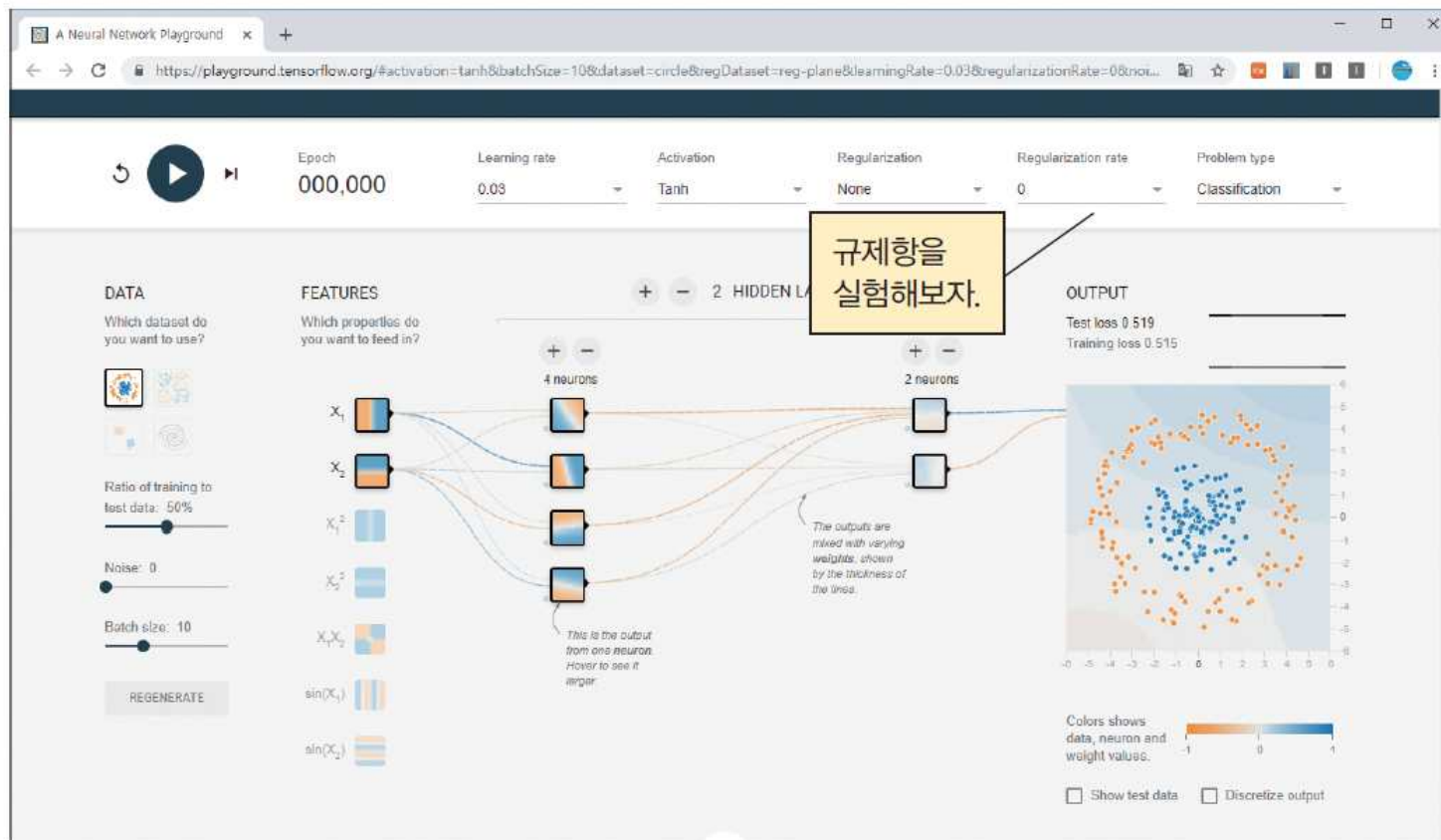


그림 14-29 드롭 아웃을 시행하면 오류가 줄어든다.

Lab: 배치 크기, 학습률, 규제항

- <https://playground.tensorflow.org>

규제항보다 학습률이 훨씬 영향이 크!

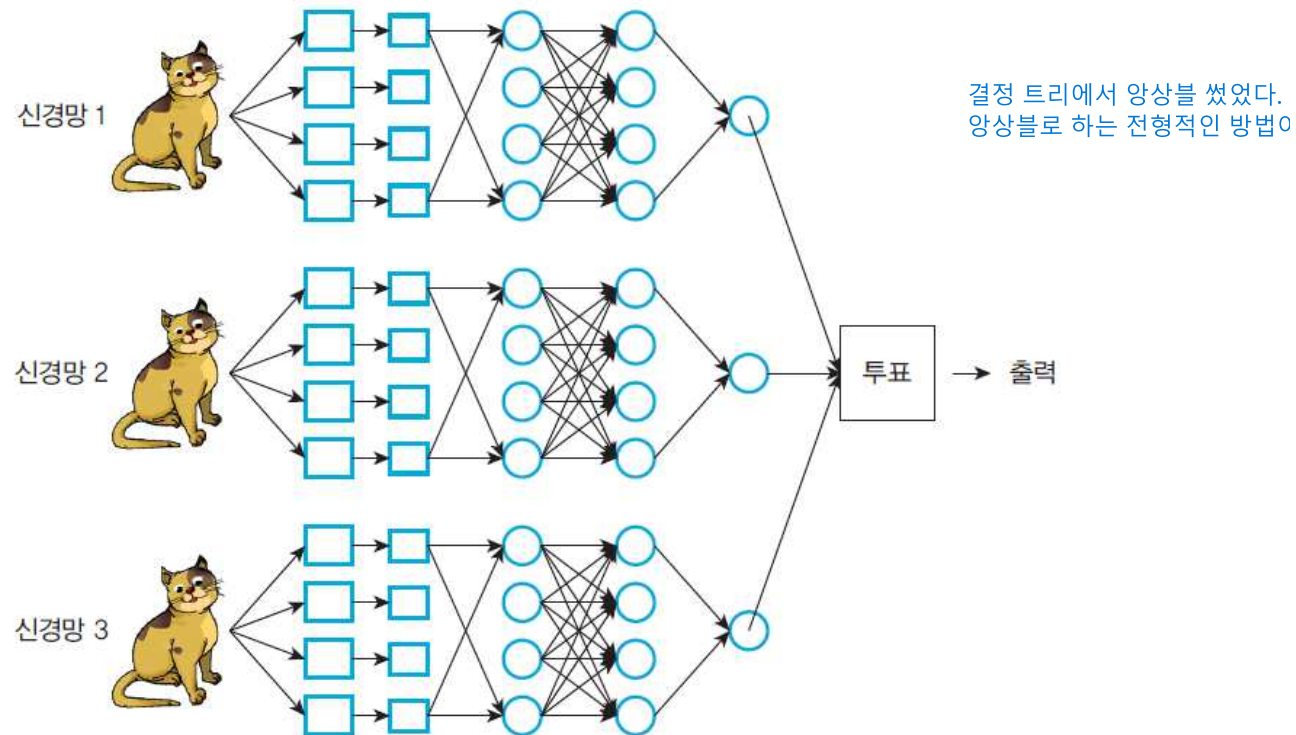


Q

- DNN에서 과잉 적합은 무엇이고 이것은 어떻게 방지할 수 있는가?

앙상블

- 앙상블(ensemble)은 여러 전문가를 동시에 훈련시키는 것과 같다. 이 방법은 동일한 딥러닝 신경망을 N 개를 만드는 것이다.
- 각 신경망을 독립적으로 학습시킨 후에 마지막에 합치는 것이다.



11 Keras를 이용한 MNIST 숫자 인식

- 우리는 이전 장에서 MNIST 숫자를 MLP로 인식해본 경험이 있다. 동일한 데이터 세트에 대하여 이번에는 심층 신경망을 사용해보자. 얼마나 정확도가 증가할까?

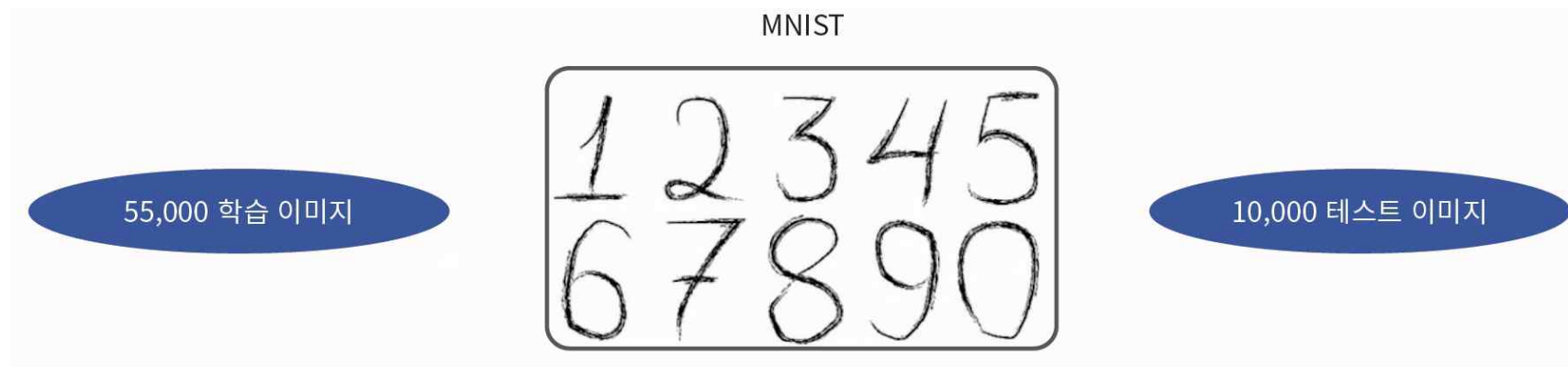


그림 14-21 MNIST 데이터

Keras를 이용한 MNIST 숫자 인식

```
import matplotlib.pyplot as plt
import tensorflow as tf

mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=(28,28)))
model.add(tf.keras.layers.Dense(512, activation='relu'))
model.add(tf.keras.layers.Dropout(0.2))
model.add(tf.keras.layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

Keras를 이용한 MNIST 숫자 인식

```
import matplotlib.pyplot as plt
import tensorflow as tf
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

plt.imshow(x_train[0], cmap="Greys");
```

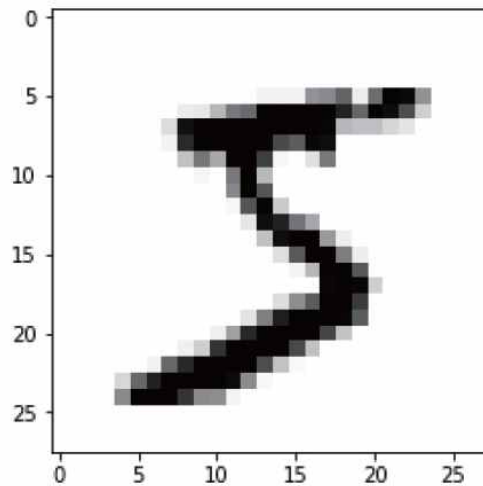


그림 14-32 매트플롯립을 사용하여 입력 이미지를 화면에 그려볼 수 있다.

Keras를 이용한 MNIST 숫자 인식

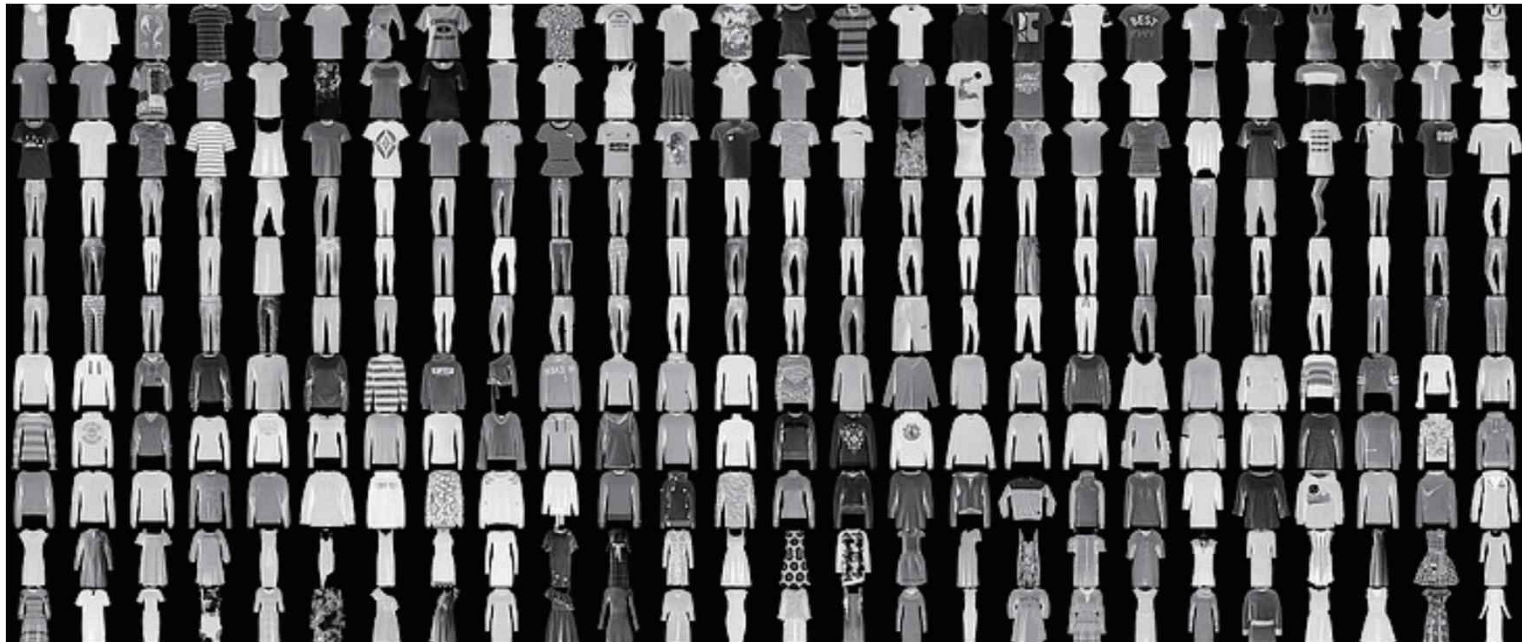
```
model = tf.keras.models.Sequential()  
model.add(tf.keras.layers.Flatten(input_shape=(28,28)))  
model.add(tf.keras.layers.Dense(512, activation='relu'))  
model.add(tf.keras.layers.Dropout(0.2))  
model.add(tf.keras.layers.Dense(10, activation='softmax'))  
  
model.compile(optimizer='adam',  
loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])  
  
model.fit(x_train, y_train, epochs=5)  
model.evaluate(x_test, y_test)
```

실행 결과

```
Epoch 1/5  
60000/60000 [=====] - 7s 116us/sample - loss: 0.2205 - acc:  
0.9348  
Epoch 2/5  
60000/60000 [=====] - 7s 110us/sample - loss: 0.0969 - acc:  
0.9700  
Epoch 3/5  
60000/60000 [=====] - 7s 109us/sample - loss: 0.0678 - acc:  
0.9785  
Epoch 4/5  
60000/60000 [=====] - 6s 108us/sample - loss: 0.0529 - acc:  
0.9834  
Epoch 5/5  
60000/60000 [=====] - 7s 108us/sample - loss: 0.0428 - acc:  
0.9859  
10000/10000 [=====] - 0s 43us/sample - loss: 0.0645 - acc: 0.9795
```

예제: 패션 아이템 분류

- 이전 절에서는 텐서플로우 튜토리얼에 나오는 패션 아이템을 심층 신경망으로 분류하는 코드를 살펴보자.
(https://www.tensorflow.org/tutorials/keras/basic_classification)



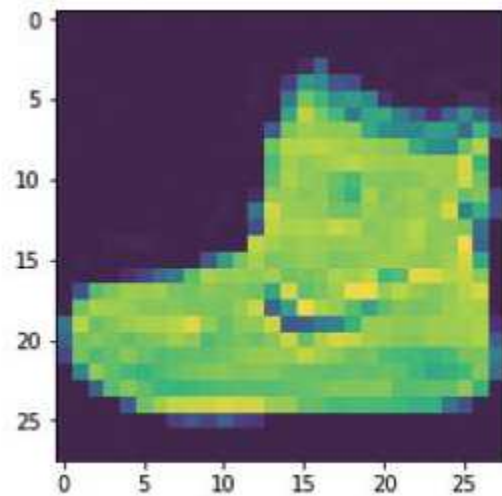
데이터 세트

- 이미지는 28x28 크기이고 픽셀 값은 0과 255 사이의 값이다. 레이블 (label)은 0에서 9까지의 정수로서 패션 아이템의 범주를 나타낸다.

레이블	범주
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

하나의 데이터

- `plt.imshow(train_images[0])`



예제: 패션 아이템 분류

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models

fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

plt.imshow(train_images[0])

train_images = train_images / 255.0
test_images = test_images / 255.0
```

예제: 패션 아이템 분류

```
model = models.Sequential()  
model.add(layers.Flatten(input_shape=(28, 28)))  
model.add(layers.Dense(128, activation='relu'))  
model.add(layers.Dense(10, activation='softmax'))  
  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
model.fit(train_images, train_labels, epochs=5)  
  
test_loss, test_acc = model.evaluate(test_images, test_labels)  
print('정확도:', test_acc)
```

```
10000/10000 [=====] - 0s 32us/sample - loss: 0.3560 -  
acc: 0.8701  
정확도: 0.8701
```

예제: 패션 아이템 분류

```
...
Epoch 1/5
60000/60000 [=====] - 4s 70us/sample - loss: 0.4988 - accuracy:
0.8264
Epoch 2/5
60000/60000 [=====] - 4s 68us/sample - loss: 0.3792 - accuracy:
0.8629
Epoch 3/5
60000/60000 [=====] - 4s 66us/sample - loss: 0.3394 - accuracy:
0.8766
Epoch 4/5
60000/60000 [=====] - 4s 66us/sample - loss: 0.3142 - accuracy:
0.8859
Epoch 5/5
60000/60000 [=====] - 4s 65us/sample - loss: 0.2977 - accuracy:
0.8907
```


Summary

- 딥러닝(Deep Learning)은 DNN(deep neural networks)에서 사용하는 학습 알고리즘이다. DNN은 MLP(다층 퍼셉트론)에서 은닉층의 개수를 증가시킨 것이다.
- 예전의 MLP에서는 인간이 영상의 특징을 추출하여서 신경망에 제공하였다. DNN에서는 특징 추출 자체도 학습으로 수행할 수 있다.
- 그라디언트 소실 문제란 은닉층이 신경망에 많이 추가되면 손실 함수의 그라디언트가 0에 가까워지고 따라서 학습이 되지 않는 현상이다. 그라디언트 소실 문제를 해결하기 위하여 DNN에서는 활성화 함수로 ReLU 함수를 많이 사용한다.
- 학습 속도를 높이기 위해서 교차 엔트로피(cross entropy)를 손실 함수로 많이 사용한다. 교차 엔트로피는 2개의 확률분포 간의 거리를 측정한 것이다.
- 가중치의 초기값은 작은 난수로 초기화되어야 한다. He 초기화
- 온라인 학습과 배치 학습의 중간에 있는 방법이 미니 배치(mini batch)이다. 이 방법에서는 훈련 데이터를 작은 배치들로 분리시켜서 하나의 배치가 끝날 때마다 학습을 수행하는 방법이다.

Q & A

